KLASIFIKASI SUARA LINGKUNGAN MENGGUNAKAN PICZAK CNN BERDASARKAN REPRESENTASI SPEKTROGRAM

TUGAS AKHIR

IGNATIUS JULIO BINTANG REGEN

121140192

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI INDUSTRI
INSTITUT TEKNOLOGI SUMATERA
LAMPUNG SELATAN
2026

LEMBAR PENGESAHAN

Saya menyatakan bahwa Tugas Akhir berjudul "Klasifikasi Suara Lingkungan Menggunakan Piczak CNN Berdasarkan Representasi Spektrogram" merupakan hasil karya saya sendiri dan belum pernah diajukan, baik sebagian maupun seluruhnya, di Institut Teknologi Sumatera atau institusi pendidikan lain oleh saya maupun pihak lain.

Lampung Selatan, 23-04-2026
Penulis,

Foto 2x3

IGNATIUS JULIO BINTANG REGEN
NIM. 121140192

Diperiksa dan disetujui oleh,

Pembimbing

1. Martin Clinton Tosima Manullang, Ph.D.
   NIP. 19930109 201903 1 017 .....

Penguji

1. Imam Eko Wicaksono, S.Si., M.Si.
   NIP. 19890517 201903 1 013 .....
2. Winda Yulita, M.Cs.
   NIP. 19930727 2022 03 2 022 .....

Disahkan oleh,
Koordinator Program Studi Teknik Informatika
Fakultas Teknologi Industri
Institut Teknologi Sumatera

Andika Setiawan, S.Kom., M.Cs.
NIP. 19911127 2022 03 1 007

HALAMAN PERNYATAAN ORISINALITAS

Tugas Akhir dengan judul "Klasifikasi Suara Lingkungan Menggunakan Piczak CNN Berdasarkan Representasi Spektrogram" adalah karya saya sendiri, dan semua sumber baik yang dikutip maupun dirujuk telah saya nyatakan benar.

Nama	**:**	**IGNATIUS JULIO BINTANG REGEN**
NIM	**:**	**121140192**
Tanda Tangan	**:**	**.....**
Tanggal	**:**	**.....**

HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI TUGAS AKHIR UNTUK KEPENTINGAN AKADEMIS

Sebagai civitas akademik Institut Teknologi Sumatera, saya yang bertanda tangan di bawah ini:

Nama : IGNATIUS JULIO BINTANG REGEN
NIM : 121140192
Program Studi : Teknik Informatika
Fakultas : Teknologi Industri
Jenis Karya : Tugas Akhir

demi pengembangan ilmu pengetahuan, menyetujui untuk memberikan kepada Institut Teknologi Sumatera **Hak Bebas Royalti Noneksklusif *(Non-exclusive Royalty Free Right)*** atas karya ilmiah saya yang berjudul:

Klasifikasi Suara Lingkungan Menggunakan Piczak CNN Berdasarkan Representasi Spektrogram

beserta perangkat yang ada (jika diperlukan). Dengan Hak Bebas Royalti Noneksklusif ini Institut Teknologi Sumatera berhak menyimpan, mengalihmedia/formatkan, mengelola dalam bentuk pangkalan data (*database*), merawat, dan memublikasikan tugas akhir saya selama tetap mencantumkan nama saya sebagai penulis/pencipta dan sebagai pemilik Hak Cipta.

Demikian pernyataan ini saya buat dengan sebenarnya.

Dibuat di : Lampung Selatan
Pada tanggal : 23-04-2026

Yang menyatakan

IGNATIUS JULIO BINTANG REGEN

KATA PENGANTAR

Puji syukur kehadirat Allah SWT/Tuhan Yang Maha Esa atas limpahan rahmat, karunia, serta petunjuk-Nya sehingga penyusunan tugas akhir ini telah terselesaikan dengan baik. Dalam penyusunan tugas akhir ini penulis telah banyak mendapatkan arahan, bantuan, serta dukungan dari berbagai pihak. Oleh karena itu pada kesempatan ini penulis mengucapan terima kasih kepada:

1. [Rozan Zaky] selaku Bos saya di Institut Teknologi Sumatera.
2. [Dekan FTI] selaku Dekan Fakultas Teknologi Industri.
3. [Koor Prodi IF] selaku Ketua Program Studi Teknik Informatika.
4. [Dosen Pembimbing] selaku Dosen Pembimbing atas ide, waktu, tenaga, perhatian, dan masukan yang telah disumbangsihkan kepada penulis.
5. [Isi nama lainnya]

Akhir kata penulis berharap semoga tugas akhir ini dapat memberikan manfaat bagi kita semua.

RINGKASAN

Klasifikasi Suara Lingkungan Menggunakan Piczak CNN Berdasarkan Representasi Spektrogram

IGNATIUS JULIO BINTANG REGEN

Perkembangan pesat di ranah kecerdasan buatan serta pembelajaran mesin telah memberikan dampak yang substansial terhadap pemrosesan sinyal digital, khususnya dalam pengenalan suara lingkungan. Studi ini mengarahkan fokusnya pada perancangan sistem klasifikasi suara lingkungan yang memanfaatkan arsitektur Piczak CNN dengan representasi spektrogram sebagai masukan, guna menunjang keselamatan serta meningkatkan kesadaran situasional penyandang gangguan pendengaran. Adapun sistem yang dikembangkan ditujukan untuk mengidentifikasi lima kategori suara yang berkaitan dengan keselamatan, meliputi klakson mobil, klakson motor, sirene, pecahan kaca, dan tangisan bayi.

Kajian literatur memperlihatkan bahwa Convolutional Neural Network (CNN) telah menunjukkan keefektifannya dalam mengklasifikasikan suara lingkungan melalui pemanfaatan representasi spektrogram maupun MFCC. Sejumlah riset sebelumnya mencatat capaian akurasi yang beragam, berkisar antara 64,5% sampai 96,23%, bergantung pada jenis arsitektur dan dataset yang dipakai. Landasan teori yang mendasari penelitian ini meliputi konsep representasi spektrogram, Mel spectrogram, arsitektur CNN, arsitektur Piczak-CNN, serta metrik evaluasi yang mencakup accuracy, precision, recall, dan F1-score.

Pada tahap metodologi, data audio dihimpun dari sumber daring berlisensi serta perekaman mandiri melalui mikrofon smartphone, yang menghasilkan total 500 sampel audio (masing-masing 100 per kelas) dengan durasi 4 detik dalam format WAV mono pada laju sampel 44.100 Hz. Proses pre-processing meliputi normalisasi amplitudo, ekstraksi fitur log-Mel

spectrogram menggunakan 60 Mel bin, segmentasi sepanjang 41 frame waktu, dan penambahan fitur delta guna menangkap dinamika temporal, sehingga menghasilkan tensor masukan berdimensi $2 \times 60 \times 41$. Teknik augmentasi data yang diterapkan mencakup time shifting, penambahan derau latar, serta spectrogram augmentation dengan probabilitas aktivasi 50% secara acak. Seluruh evaluasi dilaksanakan melalui skema 5-fold cross-validation.

Model Piczak-CNN yang dibangun memuat dua blok konvolusi (masing-masing 80 filter berukuran kernel 57×6 dan 1×3), yang tiap bloknya dilengkapi aktivasi ReLU dan max-pooling, serta dua lapisan fully connected berisi 5000 unit neuron. Pencarian hyperparameter optimal dilakukan secara terstruktur terhadap 16 kombinasi yang melingkupi variasi optimizer (SGD dan Adam), dropout rate (0,0; 0,1; 0,2; 0,3), serta penerapan augmentasi data. Hasil percobaan menunjukkan bahwa konfigurasi terbaik diperoleh dengan optimizer SGD pada learning rate 0,002, dropout rate 0,0, dan tanpa augmentasi data, yang mampu meraih akurasi rata-rata tertinggi sebesar 0,823 setelah diverifikasi melalui tiga kali percobaan.

Uji komparatif terhadap model PANNs CNN14 yang di-fine-tune pada dataset serupa memperlihatkan keunggulan Piczak-CNN, dengan akurasi rata-rata 0,836, F1-score 0,81932, dan presisi 0,8493, sementara PANNs CNN14 hanya mencapai akurasi 0,64, F1-score 0,62322, dan presisi 0,6703. Studi ablasi turut membuktikan bahwa peniadaan dropout dan augmentasi data justru meningkatkan kinerja model, sedangkan substitusi optimizer SGD dengan Adam mengakibatkan penurunan akurasi yang cukup signifikan. Dari segi efisiensi komputasional, model Piczak-CNN memiliki ukuran 205 MB dengan 26,3 juta parameter, jauh lebih ringkas dibandingkan PANNs CNN14 yang berukuran 319 MB dengan 80,7 juta parameter.

Dari keseluruhan hasil yang diperoleh, dapat disimpulkan bahwa optimalisasi arsitektur Piczak-CNN mampu mencapai akurasi 83,6% dengan F1-score 0,81932 dan presisi 84,93% dalam mengklasifikasikan lima kategori

suara peringatan keselamatan. Pemilihan optimizer, learning rate, dan dropout rate terbukti memberikan pengaruh yang signifikan terhadap capaian akurasi klasifikasi. Untuk pengembangan lebih lanjut, disarankan agar dataset diperluas, teknik augmentasi yang lebih canggih dieksplorasi, studi ablasi diperdalam guna mengevaluasi kontribusi setiap fitur dan lapisan dalam model, serta model diimplementasikan pada perangkat edge untuk menguji performa secara real-time dalam kondisi nyata.

ABSTRAK

Klasifikasi Suara Lingkungan Menggunakan Piczak CNN Berdasarkan Representasi Spektrogram

IGNATIUS JULIO BINTANG REGEN

Klasifikasi suara lingkungan sangat penting untuk peringatan keselamatan, khususnya bagi penyandang gangguan pendengaran. Namun, model deep learning saat ini umumnya membutuhkan komputasi besar sehingga memicu kendala latensi jika diterapkan pada perangkat berdaya terbatas. Penelitian ini bertujuan mengoptimalkan arsitektur Piczak CNN dalam mengklasifikasikan lima kelas suara peringatan keselamatan (klakson mobil, klakson motor, sirene, pecahan kaca, dan tangisan bayi) serta membandingkannya dengan model pre-trained berskala besar. Metodologi penelitian meliputi pengumpulan 500 sampel audio, pra-pemrosesan ekstraksi fitur log-Mel spectrogram beserta fitur delta menjadi bentuk tensor berdimensi $2 \times 60 \times 41$, dan evaluasi model menggunakan metode 5-fold cross-validation. Studi ablasi juga dilaksanakan untuk menguji pengaruh optimizer, dropout rate, dan augmentasi data. Hasil penelitian menunjukkan bahwa konfigurasi optimal Piczak CNN dicapai dengan menggunakan optimizer SGD (laju pembelajaran 0,002), tanpa dropout, dan tanpa augmentasi data. Konfigurasi tersebut menghasilkan akurasi sebesar 83,6%, yang secara signifikan terbukti mengungguli performa model pre-trained PANNs CNN14 yang hanya mencapai akurasi 64% pada dataset yang sama. Dari aspek komputasi, model Piczak CNN jauh lebih efisien dengan ukuran 205 MB dibandingkan PANNs CNN14 yang membutuhkan ruang 319 MB. Disimpulkan bahwa arsitektur model sederhana yang dioptimalkan secara spesifik terbukti lebih tangguh dan efisien dibandingkan model berskala besar dalam konteks klasifikasi dengan dataset terbatas.

Kata Kunci: Piczak-CNN, spektrogram, klasifikasi suara, keselamatan, deep learning

ABSTRACT

Environmental Sound Classification Using Piczak CNN Based on Spectrogram Representation

IGNATIUS JULIO BINTANG REGEN

Environmental sound classification is essential for safety alerts, particularly for individuals with hearing impairments. However, current deep learning models generally require substantial computational resources, leading to latency issues when deployed on resource-constrained devices. This study aims to optimize the Piczak CNN architecture for classifying five safety-related sound classes (car horn, motorcycle horn, siren, glass breaking, and baby crying) and to compare its performance against a large-scale pre-trained model. The research methodology involves collecting 500 audio samples, preprocessing through log-Mel spectrogram feature extraction along with delta features to produce a tensor of dimensions $2 \times 60 \times 41$, and evaluating the model using 5-fold cross-validation. An ablation study was also conducted to examine the effects of optimizer selection, dropout rate, and data augmentation. The results demonstrate that the optimal Piczak CNN configuration is achieved using the SGD optimizer (learning rate of 0.002), without dropout, and without data augmentation. This configuration yields an accuracy of 83.6%, significantly outperforming the pre-trained PANNs CNN14 model, which only achieves an accuracy of 64% on the same dataset. In terms of computational efficiency, the Piczak CNN model is considerably more compact at 205 MB compared to 319 MB for PANNs CNN14. It is concluded that a simple architecture optimized specifically for a given domain proves to be more robust and efficient than a large-scale model in the context of classification with a limited dataset.

Keywords: Piczak-CNN, spectrogram, sound classification, safety, deep learning

DAFTAR ISI

LEMBAR PENGESAHAN . . . . . **ii**

HALAMAN PERNYATAAN ORISINALITAS . . . . . **iii**

HALAMAN PERSETUJUAN PUBLIKASI . . . . . **iv**

KATA PENGANTAR . . . . . **v**

RINGKASAN . . . . . **vi**

ABSTRAK . . . . . **ix**

ABSTRACT . . . . . **x**

DAFTAR ISI . . . . . **xi**

DAFTAR TABEL . . . . . **xiv**

DAFTAR GAMBAR . . . . . **xv**

DAFTAR RUMUS . . . . . **xvi**

DAFTAR KODE . . . . . **xvii**

BAB I PENDAHULUAN **1**

1.1 Latar Belakang 1

1.2 Rumusan Masalah 2

1.3 Tujuan Penelitian 3

1.4 Batasan Masalah 3

1.5 Manfaat Penelitian 4

1.6 Sistematika Penulisan 4

1.6.1 Bab I 4

1.6.2 Bab II 4

1.6.3 Bab III 4

1.6.4 Bab IV 5
1.6.5 Bab V 5

BAB II TINJAUAN PUSTAKA 6

2.1 Tinjauan Pustaka 6
2.2 Dasar Teori 12
2.2.1 Representasi Spektrogram 12
2.2.2 *Mel Spectrogram* 12
2.2.3 Convolutional Neural Network (CNN) 13
2.2.4 PiczakCNN 14
2.2.5 PANNs CNN14 15
2.2.5.1 Layer Architecture 16
2.2.6 *Max-Pooling* 16
2.2.6.1 Rectified Linear Units 16
2.2.6.2 Dropout Learning 17
2.2.7 Confusion Matrix 17
2.2.7.1 Accuracy 18
2.2.7.2 Precision 19
2.2.7.3 Recall 19
2.2.7.4 F1-Score 19

BAB III METODE PENELITIAN 20

3.1 Alur Penelitian 20
3.2 Penjabaran Langkah Penelitian 21
3.2.1 Identifikasi Masalah 21
3.2.2 Studi Literatur 21
3.2.3 Pengumpulan Data 21
3.2.4 Pengembangan Model 22
3.2.5 Evaluasi Model 22
3.2.6 Hasil dan Pembahasan 23

3.3 Alat dan Bahan Tugas Akhir 23
3.3.1 Alat 23
3.3.2 Bahan 23
3.4 Metode Pengembangan Model 24
3.4.1 *Input Dataset* 25
3.4.2 *Data Labeling* 25
3.4.3 *Split* Data 25
3.4.4 *Pre-processing* Data 26
3.4.4.1 *Augmentation* 26
3.4.4.2 *Feature Extraction* 30
3.4.4.3 *Normalization* dan *Padding* 30
3.4.4.4 *Segmentation* 31
3.4.4.5 *Delta Features* 31
3.4.5 Pelatihan Model 31
3.4.6 Lapisan Input 32
3.4.6.1 Lapisan Konvolusi Pertama (Conv1) 33
3.4.6.2 Lapisan Max Pooling Pertama (MaxPool1) 33
3.4.6.3 Lapisan Max Pooling Kedua (MaxPool2) 33
3.4.6.4 Lapisan Fully Connected dan Regularisasi 33
3.4.6.5 Lapisan Output 33
3.4.7 *Hyperparameter* Tuning 34
3.5 Ilustrasi Perhitungan 36
3.6 Rencana Pengujian 38
3.6.1 Studi Ablasi 38
3.6.1.1 Konsep Studi Ablasi 38
3.6.1.2 Skenario Pengujian 38
BAB IV HASIL DAN PEMBAHASAN 40
4.1 Hasil 40

4.1.1 Pengumpulan Data 40
4.1.2 Split Data 40
4.1.3 *Pre-Processing* dan Augmentasi Data 41
4.1.3.1 *Pre-Processing* Data 41
4.1.3.2 Augmentasi Data 46
4.1.4 Pelatihan Model 47
4.1.4.1 Pengujian Model 53
4.1.4.2 Perbandingan Model 55
4.1.5 Evaluasi Studi Ablasi 60
4.1.5.1 Baseline Model 60
4.1.5.2 Eksperimentasi Dropout 61
4.1.5.3 Eksperimentasi Augmentasi Data 61
4.1.5.4 Eksperimentasi Optimizer 61
4.2 Evaluasi 61
4.3 Pembahasan 63

BAB V KESIMPULAN DAN SARAN 66
5.1 Kesimpulan 66
5.2 Saran 67

DAFTAR PUSTAKA 68

LAMPIRAN 72
A GitHub Proyek Tugas Akhir 72

DAFTAR TABEL

Tabel 2.1 Literasi Penelitian Terdahulu 9
Tabel 2.2 Confusion Matrix 18
Tabel 2.3 Multiclass Confusion Matrix (5 Kelas) 18
Tabel 3.1 Data Labeling 25
Tabel 3.2 *Hyperparameter* Default Model 34
Tabel 3.3 *Hyperparameter* Studi Ablasi 36
Tabel 3.4 Contoh *Confusion Matrix* dengan Struktur Header 37
Tabel 4.1 Konfigurasi Hyperparameter Pelatihan Model 51
Tabel 4.2 Hasil Eksperimen Pemilihan *Hyperparameter* 53
Tabel 4.3 Hasil Verifikasi 3 Kombinasi *Hyperparameter* Terbaik 55
Tabel 4.4 Perbandingan Arsitektur Model Piczak-CNN dan PANNs CNN14 58
Tabel 4.5 Perbandingan Performa Model Piczak-CNN dan PANNs CNN14 59

DAFTAR GAMBAR

Gambar 2.1 Mel Spectrogram 13
Gambar 2.2 Arsitektur Convolutional Neural Network (CNN) 14
Gambar 2.3 Arsitektur PiczakCNN 14
Gambar 3.1 Alur Penelitian 20
Gambar 3.2 Metode Pengembangan Model 24
Gambar 3.3 Alur *Pre-processing* Data 26
Gambar 3.4 Sebelum *Time Shifting* 27
Gambar 3.5 Sesudah *Time Shifting* 27
Gambar 3.6 Sebelum Penambahan Derau 28
Gambar 3.7 Sesudah Penambahan Derau 28
Gambar 3.8 Sebelum *Spectrogram Augmentation* 29
Gambar 3.9 Sesudah *Spectrogram Augmentation* 29
Gambar 3.10 Arsitektur *Piczak-CNN* [10] 32
Gambar 4.1 Alur Pipeline Pre-processing Data Audio 42
Gambar 4.2 Perbandingan Log-Mel Spectrogram 42
Gambar 4.3 Klakson Mobil 43
Gambar 4.4 Klakson Motor 43
Gambar 4.5 Pecahan Kaca 44
Gambar 4.6 Sirine 44
Gambar 4.7 Tangisan Bayi 45

DAFTAR RUMUS

Rumus 2.1 Kompleksitas komputasi lapisan konvolusi 2D 15
Rumus 2.2 Kompleksitas komputasi lapisan fully connected 15
Rumus 2.3 Fungsi aktivasi ReLU 16
Rumus 2.4 Rumus perhitungan accuracy 18
Rumus 2.5 Rumus perhitungan precision 19
Rumus 2.6 Rumus perhitungan recall 19
Rumus 2.7 Rumus perhitungan F1-score 19
Rumus 3.1 Perhitungan *accuracy* 37
Rumus 3.2 Perhitungan *precision* 37
Rumus 3.3 Perhitungan *recall* 37
Rumus 3.4 Perhitungan F1-*score* 37

DAFTAR KODE

Kode 4.1 Implementasi kode 5-fold-cross-validation 41
Kode 4.2 Implementasi kode pre-processing data audio 45
Kode 4.3 Implementasi Augmentasi Data 46
Kode 4.4 Arsitektur Model Piczak-CNN 49
Kode 4.5 Implementasi Pelatihan Model 51
Kode 4.6 Arsitektur Model PANNs CNN14 56

BAB I
PENDAHULUAN

1.1 Latar Belakang

Kemajuan pesat pada bidang kecerdasan buatan (*Artificial Intelligence*) dan pembelajaran mesin (*Machine Learning*) telah mendorong peningkatan signifikan dalam pemrosesan sinyal digital, termasuk klasifikasi suara lingkungan (*Environmental Sound Classification*). Suara lingkungan merupakan bagian penting dari aktivitas sehari-hari yang memuat informasi mengenai aktivitas manusia dan kondisi sekitar. Seiring meningkatnya urbanisasi dan kebisingan perkotaan, kemampuan untuk memilah serta mengidentifikasi suara-suara krusial di tengah keramaian menjadi tantangan teknis yang kompleks [1].

Penelitian ini berfokus pada aspek keselamatan publik, khususnya bagi penyandang gangguan pendengaran (Teman Tuli dan Tunarungu). Bagi individu dengan pendengaran normal, suara peringatan seperti klakson kendaraan, sirene ambulans, atau suara kaca pecah merupakan indikator bahaya yang segera dikenali. Sebaliknya, bagi penyandang gangguan pendengaran, keterbatasan dalam mendeteksi sinyal auditif tersebut dapat menimbulkan risiko tinggi, terutama saat beraktivitas di jalan raya atau di lingkungan domestik [2], [3], [4], [5], [6], [7]. Oleh karena itu, diperlukan sebuah sistem cerdas yang berfungsi sebagai telinga digital untuk mendeteksi suara bahaya secara *real-time* dengan tingkat akurasi yang memadai.

Meskipun metode *Deep Learning* modern seperti *Audio Spectrogram Transformer (AST)* dan *Convolutional Recurrent Neural Network (CRNN)* menunjukkan akurasi tinggi, model-model tersebut umumnya membutuhkan sumber daya komputasi dan memori besar [8]. Penerapan model berkompleksitas tinggi pada perangkat bergerak (*mobile*) atau *Internet of*

Things (IoT) sering menimbulkan latensi yang tidak dapat ditoleransi dalam situasi darurat. Dalam konteks keselamatan, keterlambatan pada proses deteksi bahkan dalam skala milidetik dapat berdampak signifikan. Oleh sebab itu, efisiensi komputasi menjadi faktor kunci agar sistem dapat berjalan optimal pada perangkat *edge* dan *Edge AI* [9].

Menanggapi kebutuhan tersebut, penelitian ini mengadopsi dan mengoptimalkan arsitektur Piczak *CNN*. Meskipun diperkenalkan oleh Piczak (2015) sebagai salah satu pendekatan awal penggunaan *CNN* pada spektrogram [10], karakteristiknya yang ringan dengan jumlah parameter relatif kecil menjadikan arsitektur ini relevan untuk penerapan *Edge AI* saat ini [11]. Piczak *CNN* menawarkan keseimbangan antara efisiensi komputasi dan akurasi. Dengan memanfaatkan representasi suara berupa *Log-Mel Spectrogram* yang dilengkapi fitur *Delta* untuk menangkap dinamika temporal, model dirancang agar dapat beroperasi efisien pada perangkat berdaya terbatas seperti ponsel pintar [12].

Secara spesifik, penelitian ini bertujuan mengimplementasikan arsitektur Piczak *CNN* untuk mendeteksi lima kelas suara prioritas keselamatan, yakni klakson mobil, klakson motor, sirene, kaca pecah, dan tangisan bayi. Pemilihan kelas tersebut didasarkan pada literatur klasifikasi suara lingkungan dan audio surveillance yang menempatkan kelas-kelas ini sebagai *safety-critical sounds* untuk konteks lalu lintas, keamanan lingkungan, dan pemantauan domestik [13], [14], [15], [16]. Diharapkan sistem yang dikembangkan mampu memberikan akurasi tinggi, ketangguhan terhadap gangguan *noise*, serta efisiensi eksekusi secara *real-time* guna mendukung peningkatan kesadaran situasional dan keselamatan bagi penyandang gangguan pendengaran.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan di atas, maka permasalahan penelitian dirumusukan sebagai berikut:

1. Bagaimana mengoptimalkan arsitektur Piczak *CNN* untuk klasifikasi suara lingkungan yang berorientasi pada keselamatan?
2. Bagaimana perbandingan performa model Piczak *CNN* dengan model pre-trained berskala besar dalam hal akurasi dan efisiensi komputasional pada dataset suara lingkungan dengan ukuran terbatas?

1.3 Tujuan Penelitian

Berdasarkan rumusan masalah yang telah diuraikan di atas, maka tujuan dari penelitian ini adalah :

1. Mengoptimalkan arsitektur Piczak *CNN* ke dalam sistem klasifikasi suara lingkungan yang difokuskan pada deteksi suara peringatan keselamatan.
2. Membandingkan performa model Piczak *CNN* dengan model pre-trained berskala besar dalam hal akurasi dan efisiensi komputasional pada dataset suara lingkungan dengan ukuran terbatas.

1.4 Batasan Masalah

Adapun batasan masalah dari penelitian ini agar sesuai dengan yang diharapkan adalah sebagai berikut:

1. Penelitian ini tidak mengembangkan arsitektur *CNN* baru, melainkan berfokus pada implementasi dan optimasi arsitektur Piczak *CNN* yang telah dipublikasikan.
2. Data yang digunakan merupakan *dataset* suara lingkungan yang diperoleh melalui perekaman menggunakan mikrofon pada perangkat ponsel.
3. Ruang lingkup klasifikasi dibatasi pada lima kelas suara berorientasi keselamatan yaitu klakson mobil, klakson motor, sirene, kaca pecah, dan tangisan bayi.

1.5 Manfaat Penelitian

Adapun manfaat yang diperoleh dari hasil penelitian ini adalah sebagai berikut:

1. Memberikan pemahaman yang lebih mendalam tentang penerapan arsitektur Piczak *CNN* untuk klasifikasi suara lingkungan.
2. Menjadi referensi bagi penelitian selanjutnya dalam bidang klasifikasi suara lingkungan menggunakan metode *deep learning*.
3. Memberikan kontribusi dalam pengembangan sistem klasifikasi suara lingkungan yang lebih akurat dan efisien.

1.6 Sistematika Penulisan

Sistematika penulisan berisi pembahasan apa yang akan ditulis disetiap Bab. Sistematika pada umumnya berupa paragraf yang setiap paragraf mencerminkan bahasan setiap Bab.

1.6.1 Bab I

Bab ini berisikan penjelasan latar belakang dari topik penelitian yang berlangsung, rumusan masalah dari masalah yang dihadapi pada penjelasan di latar belakang, tujuan dari penelitian, batasan dari penelitian, manfaat dari hasil penelitian, dan sistematika penulisan tugas akhir.

1.6.2 Bab II

Bab ini membahas mengenai tinjauan pustaka dari penelitan terdahulu dan dasar teori yang berkaitan dengan penelitian ini.

1.6.3 Bab III

Bab ini berisikan penjelasan alur kerja sistem, alat dan data yang digunakan, metode yang digunakan, dan rancangan pengujian.

1.6.4 Bab IV

Bab ini membahas hasil implementasi dan pengujian dari penelitian yang dilakukan, serta analisis dan evaluasi yang dapat dipetik dari hasil.

1.6.5 Bab V

Bab ini membahas kesimpulan dari hasil penelitian dan juga saran untuk penelitian selanjutnya.

BAB II
TINJAUAN PUSTAKA

2.1 Tinjauan Pustaka

Penelitian ini ditopang oleh kajian literatur dari beragam sumber yang relevan, yang dirangkum pada Tabel 2.1. Karya PiczakCNN (2015) berjudul *Environmental Sound Classification with Convolutional Neural Networks* menekankan kesulitan mengklasifikasikan suara lingkungan yang heterogen dan saling tumpang tindih. Pendekatan yang diambil memanfaatkan ekstraksi *Mel-Frequency Cepstral Coefficients* (*MFCC*) dan klasifikasi menggunakan *Convolutional Neural Network* (*CNN*). *Dataset* yang dipakai adalah *UrbanSound8K* dengan 8.732 cuplikan *audio* berdurasi hingga 4 detik dari 10 kelas. Eksperimen melaporkan akurasi uji 73,7% serta mengungguli sejumlah metode *machine learning* konvensional, mengukuhkan keampuhan *CNN* untuk klasifikasi suara lingkungan yang bervariasi dan kompleks[10].

Penelitian oleh Wahyuningtyas (2021) berjudul *Implementasi Ekstraksi Fitur untuk Klasifikasi Suara Urban Menggunakan Deep Learning* berfokus pada perolehan representasi fitur *audio* yang informatif guna meningkatkan akurasi klasifikasi suara perkotaan. Metode yang digunakan mencakup ekstraksi fitur *MFCC* dan klasifikasi berbasis *CNN*. Evaluasi menunjukkan kinerja tinggi dengan akurasi pelatihan 91,96% dan akurasi pengujian 91,92%. Kelas dengan prediksi tertinggi adalah *Drilling* (99,59%), disusul *Street Music* (96,92%), *Air Conditioner* (90,66%), dan *Dog Bark* (84,29%). Hal ini mengindikasikan kombinasi *MFCC* dan *CNN* efektif untuk pengenalan suara urban[17].

Penelitian Hendra Kusumah dkk. (2023) berjudul *Klasifikasi Kendaraan Berbasis Suara di Lalu Lintas: Implementasi TinyML* mengembangkan sistem deteksi suara kendaraan untuk menunjang keselamatan pengendara

tunarungu, mengingat tingginya angka kecelakaan lalu lintas yang salah satunya dipicu keterbatasan persepsi auditori. Perangkat yang digunakan adalah *Arduino Nano 33 BLE Sense* dengan mikrofon terintegrasi, sedangkan pelatihan model dilakukan di *Edge Impulse*. Arsitektur yang diadopsi berupa jaringan saraf ringan dengan empat lapisan konvolusi 1-D dan satu lapisan tersembunyi, memanfaatkan fitur *Mel-Filterbank Energy* (*MFE*) dan *spektrogram*. Eksperimen menunjukkan akurasi latih 96,3% dan uji 90,9%, dengan *F1-score* di atas 95%. Terdapat kebingungan klasifikasi motor vs truk sebesar 19,5%; meskipun demikian, model dinilai andal serta layak diimplementasikan untuk mendukung keselamatan pengendara dengan gangguan pendengaran[18].

Penelitian Anam Bansal dkk. (2022) berjudul *Urban Sound Classification Using Convolutional Neural Networks* menyoroti kesulitan membedakan beragam suara perkotaan yang kompleks, seperti suara kendaraan, kebisingan konstruksi, dan aktivitas manusia yang sering tumpang tindih. Metodenya meliputi ekstraksi *MFCC* serta klasifikasi menggunakan *CNN*. *Dataset* yang digunakan adalah *UrbanSound8K* yang berisi 8.732 klip *audio* berdurasi maksimal 4 detik. Hasil menunjukkan model *CNN* mampu mencapai akurasi sekitar 87%, dan melampaui beberapa pendekatan tradisional berbasis *machine learning*, sehingga menguatkan efektivitas *CNN* untuk klasifikasi suara urban[19].

Penelitian oleh Fatih Demir dkk. (2020) berjudul *A New Deep CNN Model for Environmental Sound Classification* menyoroti keterbatasan model *CNN* pra-latih (*pre-trained*) yang pada awalnya dirancang untuk citra sehingga kurang ideal menangkap karakteristik suara lingkungan. Studi ini mengusulkan arsitektur *CNN* baru yang lebih ringan, dilatih *end-to-end* dari citra *spektrogram*, dan dikombinasikan dengan metode klasifikasi *Random Subspace K-Nearest Neighbor* (*KNN*) *ensembles*. *Dataset* yang digunakan adalah *DCASE-2017 ASC* (6.300 *audio* berdurasi 10 detik, 15 kelas) dan *UrbanSound8K* (8.732

audio berdurasi hingga 4 detik, 10 kelas). Hasil menunjukkan akurasi 96,23% pada *DCASE-2017* dan 86,70% pada *UrbanSound8K*, melampaui beberapa pendekatan *CNN* pra-latih. Temuan ini menegaskan efektivitas gabungan *CNN end-to-end* dan *KNN ensembles* untuk klasifikasi suara lingkungan[1].

Penelitian oleh Baljinder Kaur dan Jaskirat Singh (2021) berjudul *Audio Classification: Environmental Sounds Classification* menggarisbawahi tantangan mengklasifikasikan aneka suara lingkungan perkotaan yang kerap saling tumpang tindih, sehingga diperlukan sistem yang lebih akurat dan efisien. Studi ini memanfaatkan *dataset UrbanSound8K* berisi 8.732 cuplikan *audio* berdurasi kurang dari 4 detik dari 10 kelas. Pendekatan yang diterapkan meliputi *MFCC*, *CNN*, dan eksplorasi arsitektur *SincNet* untuk pemrosesan sinyal mentah. Evaluasi menghasilkan akurasi pelatihan 68,15% dan validasi 76,36%, dengan *loss* pelatihan 0,9385 dan validasi 0,7294. Ini menunjukkan kombinasi *MFCC*, *CNN*, dan *SincNet* berpotensi meningkatkan kinerja, walau akurasinya masih perlu ditingkatkan pada riset lanjutan[20].

Ringkasan tinjauan pustaka disajikan setelah uraian masing-masing jurnal.

Tabel 2.1 Literasi Penelitian Terdahulu

No.	**Judul**	**Masalah**	**Metode**	**Hasil**
1	*Environmental Sound Classification with Convolutional Neural Networks*	Tantangan klasifikasi suara lingkungan pada *dataset ESC* (kelas beragam dan sinyal saling tumpang tindih)	Masukan *log-mel spectrogram* (dengan fitur *delta*), klasifikasi berbasis *CNN*	Akurasi ≈65% (*ESC–50*) dan 80% (*ESC–10*)
2	Implementasi Ekstraksi Fitur untuk Klasifikasi Suara Urban Menggunakan Deep Learning	Variasi dan tumpang tindih suara lingkungan perkotaan yang kompleks	Ekstraksi *MFCC* dan klasifikasi *CNN*	Akurasi latih 92%, uji 92%; kelas *Drilling* 99.6%

No.	Judul	Masalah	Metode	Hasil
3	Klasifikasi Kendaraan Berbasis Suara di Lalu Lintas: Implementasi *TinyML*	Risiko kecelakaan tinggi bagi pengendara tunarungu karena kesulitan mengenali suara kendaraan	*TinyML* berbasis *CNN* 1-D dan fitur *MFE*	Latih 96%, uji 91%, *F1* ≈95%; kebingungan motor–truk ≈20%
4	*Urban Sound Classification Using Convolutional Neural Networks*	Kesulitan membedakan suara perkotaan yang sering tumpang tindih	Ekstraksi *MFCC* dan klasifikasi *CNN*	Akurasi ≈87%
5	*A New Deep CNN Model for Environmental Sound Classification*	Keterbatasan *CNN* pra-latih untuk menangkap karakteristik suara lingkungan	Arsitektur *CNN end-to-end* dipadukan *KNN ensembles*	Akurasi 96% (*DCASE–2017*) dan 87% (*UrbanSound8K*)

No.	Judul	Masalah	Metode	Hasil
6	*Audio Classification: Environmental Sounds Classification*	Klasifikasi suara lingkungan tetap menantang karena kompleksitas suara urban	*MFCC*, *CNN*, dan eksplorasi *SincNet*	Akurasi latih 68%, validasi 76%; *loss* 0,94 / 0,73

Merujuk Tabel 2.1, sebagian besar studi mengandalkan *CNN* pada representasi *spektrogram*/*MFCC*. Piczak (2015) memakai masukan *log-mel spectrogram* (ditambah fitur *delta*) dan *CNN*, dengan capaian sekitar 64,5% pada *ESC–50* dan 80% pada *ESC–10*. Wahyuningtyas (2021) mengombinasikan *MFCC* dan *CNN* dengan akurasi latih 91,96% serta uji 91,92% (tertinggi pada *Drilling* 99,59%). Hendra Kusumah dkk. (2023) menerapkan *TinyML* (*CNN* 1-D) dengan *MFE*; akurasi latih 96,3%, uji 90,9%, *F1* di atas 95%, namun ada kebingungan motor–truk 19,5%. Bansal dkk. (2022) menggunakan *MFCC+CNN* dan mencapai sekitar 87%. Demir dkk. (2020) mengajukan *CNN end-to-end* dipadukan *KNN ensembles*, meraih 96,23% pada *DCASE-2017* dan 86,70% pada *UrbanSound8K*. Kaur dan Singh (2021) mengeksplorasi *MFCC*, *CNN*, dan *SincNet*, dengan akurasi latih 68,15% dan validasi 76,36% (*loss* 0,9385/0,7294).

Dengan demikian, walaupun *CNN* efektif untuk klasifikasi suara lingkungan, masih terdapat keterbatasan seperti perbedaan kualitas/karakteristik *dataset*, akurasi yang belum konsisten tinggi di seluruh studi, serta kebingungan pada kelas yang serupa.

2.2 Dasar Teori

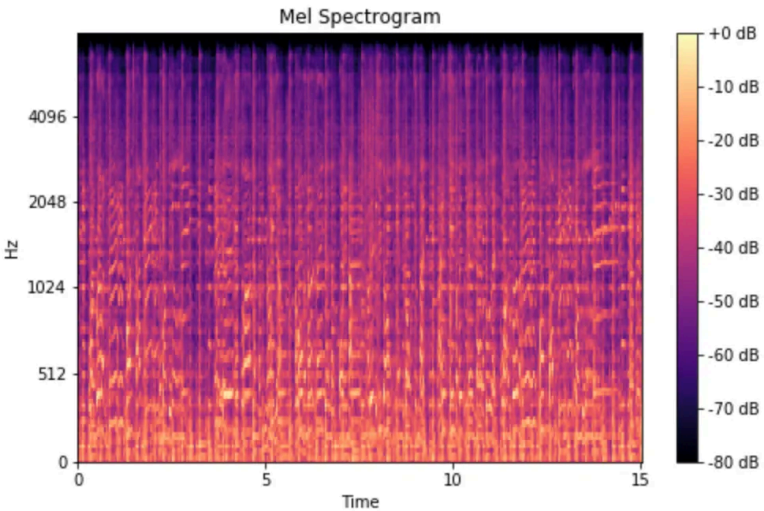
Dasar teori pada penelitian ini menjadi landasan konseptual yang merangkum konsep, definisi, dan prinsip-prinsip teoretis relevan dengan ruang lingkup studi. Bagian ini menyediakan kerangka untuk memahami fenomena yang dikaji sekaligus menjadi acuan perumusan metodologi, pelaksanaan analisis, dan penarikan kesimpulan.

2.2.1 Representasi Spektrogram

Spektrogram adalah representasi dua dimensi yang menggambarkan perkembangan spektrum terhadap waktu: satu sumbu menunjukkan waktu, sumbu lain frekuensi, sementara tingkat kecerahan/warna merepresentasikan amplitudo/energi komponen frekuensi pada tiap bingkai waktu. Karena berbentuk citra, banyak arsitektur *CNN* yang umum di visi komputer dapat langsung diterapkan pada sinyal *audio*[21].

2.2.2 *Mel Spectrogram*

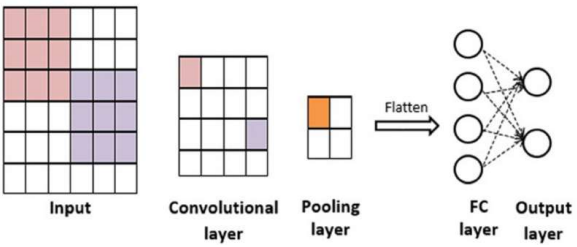
Mel Spectrogram merupakan *spektrogram* yang diproyeksikan ke skala *mel*, yakni skala frekuensi yang menyesuaikan persepsi pendengaran manusia. *MelSpec* ditampilkan sebagai citra yang memetakan variasi amplitudo terhadap waktu pada setiap frekuensi. Skala *mel* memadatkan rentang frekuensi rendah dan meregangkan frekuensi tinggi, sehingga lebih selaras dengan cara manusia menangkap suara[22].



Gambar 2.1 Mel Spectrogram

2.2.3 Convolutional Neural Network (CNN)

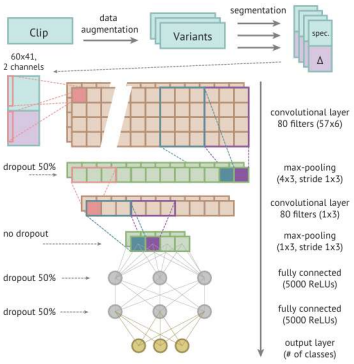
Convolutional Neural Network (*CNN*) merupakan arsitektur jaringan saraf yang dirancang untuk memproses data berdimensi dua (misalnya citra) melalui operasi *convolution* menggunakan *filter* (*kernel*) guna mengekstraksi fitur secara bertingkat. Secara struktural, *CNN* memiliki bobot, *bias*, dan fungsi *aktivasi sebagaimana* jaringan saraf pada umumnya, namun dilengkapi dengan lapisan *convolution* serta *pooling* yang berperan mereduksi dimensi sambil mempertahankan informasi yang esensial. Proses pelatihan dilakukan menggunakan *backpropagation* dan optimisasi berbasis gradien, sedangkan pada tahap inferensi digunakan *forward pass* untuk menghasilkan prediksi. *CNN* banyak diimplementasikan dalam pengenalan citra dan dapat diadaptasi untuk pemrosesan *audio* melalui representasi *spektrogram*/*Mel-spektrogram*[23]. Berikut contoh gambar arsitektur *CNN* pada Gambar 2.2 [24].



Gambar 2.2 Arsitektur Convolutional Neural Network (CNN)

2.2.4 PiczakCNN

PiczakCNN adalah arsitektur *CNN* dengan dua blok *convolution* yang masing-masing diikuti *max-pooling*, kemudian dua lapisan *fully connected*[10]. Desain ini memfasilitasi ekstraksi fitur dari level rendah hingga tinggi sekaligus menekan *overfitting* melalui *pooling* dan *regularisasi*. Pemotongan *spektrogram* serta penambahan fitur *delta* memperkaya pemodelan pola temporal dan spektral pada suara lingkungan yang cenderung bervariasi dan berisik. Selain itu, *augmentasi data* dan *segmentasi spektrogram* diterapkan untuk meningkatkan keandalan serta generalisasi model pada *dataset* beragam. Berikut merupakan gambar arsitektur *PiczakCNN* pada Gambar 2.3 [10].



Gambar 2.3 Arsitektur PiczakCNN

2.2.5 PANNs CNN14

PANNs CNN14 adalah arsitektur *Convolutional Neural Network* yang dirancang untuk tugas *audio tagging* dan klasifikasi suara lingkungan. Model ini memproses representasi waktu-frekuensi (umumnya *log-mel spectrogram*) secara *end-to-end*, sehingga mampu mempelajari pola spektral dan temporal secara bertingkat dari fitur tingkat rendah hingga fitur semantik tingkat tinggi[25].

Secara umum, *CNN14* dibangun dari rangkaian blok *convolution* yang disertai normalisasi, fungsi aktivasi nonlinier, dan *pooling*. Jumlah kanal fitur meningkat bertahap (misalnya 32, 64, 128, 128, 256, 512) untuk memperkaya kapasitas representasi. Pada bagian akhir, agregasi fitur global dan lapisan *fully connected* digunakan untuk memetakan representasi fitur ke ruang kelas keluaran[25].

Dari sisi teori komputasi, biaya utama model *CNN* berasal dari operasi konvolusi. Kompleksitas komputasi lapisan *convolution* 2D dinyatakan sebagai:

$$O(H \cdot W \cdot C_{\text{in}} \cdot C_{\text{out}} \cdot K_h \cdot K_w)$$

(Rumus 2.1)

dengan H dan W sebagai dimensi peta fitur, C_{in} dan C_{out} jumlah kanal masuk-keluar, serta K_h dan K_w ukuran kernel. Adapun kompleksitas lapisan *fully connected* adalah:

$$O(N_{\text{in}} \cdot N_{\text{out}})$$

(Rumus 2.2)

dengan N_{in} dan N_{out} sebagai jumlah neuron masukan dan keluaran. Secara konseptual, peningkatan kedalaman jaringan dan jumlah kanal pada *PANNs CNN14* cenderung meningkatkan kemampuan ekstraksi fitur, tetapi juga meningkatkan kebutuhan memori dan komputasi[25].

2.2.5.1 Layer Architecture

Arsitektur lapisan pada *CNN* umumnya terdiri dari lapisan *input*, beberapa blok *convolution–activation–pooling*, lalu diakhiri lapisan *fully connected* untuk pemetaan ke kelas *output*. Lapisan *convolution* memakai sejumlah *kernel* untuk mengekstraksi fitur lokal pada representasi *audio* (misalnya *Mel-spektrogram*); contoh konfigurasi mencakup 80 *filter* berukuran 57×6 atau 1×3 dengan pengaturan *stride*/*padding* tertentu. Lapisan *pooling* (misalnya *max-pooling* 4×3 atau 1×3) mereduksi resolusi peta fitur sekaligus meningkatkan invariansi terhadap translasi[10].

2.2.6 *Max-Pooling*

Max-pooling adalah teknik *down-sampling* pada jaringan saraf konvolusional (*CNN*) yang mereduksi dimensi spasial peta fitur dengan memilih nilai maksimum dalam wilayah tertentu (misalnya 2x2 atau 3x3). Tujuannya adalah mengurangi jumlah parameter dan komputasi, sekaligus mempertahankan fitur paling menonjol. Dengan demikian, *max-pooling* membantu mengendalikan *overfitting* dan meningkatkan kemampuan generalisasi model terhadap data baru[23].

2.2.6.1 Rectified Linear Units

Rectified Linear Unit (*ReLU*) adalah fungsi *aktivasi* nonlinier pada jaringan saraf, termasuk *CNN*, yang didefinisikan sebagai:

$$f(x) = \max(0, x)$$

(Rumus 2.3)

Dibandingkan *sigmoid* atau *hyperbolic tangent*, *ReLU* lebih efisien secara komputasi dan memungkinkan propagasi gradien yang lebih stabil, sehingga pelatihan lebih cepat dan risiko *vanishing gradient* berkurang. *ReLU* juga menghasilkan *aktivasi sparse* hanya sebagian neuron aktif yang meningkatkan efisiensi dan fokus pada fitur relevan. Namun, *ReLU* berpotensi menimbulkan

neuron “mati” saat output selalu nol; karenanya, varian seperti *Leaky ReLU* diperkenalkan. Dalam pemrosesan suara lingkungan, *ReLU* membantu jaringan mempelajari pola kompleks secara efisien dan mendukung kinerja klasifikasi tinggi[10].

2.2.6.2 Dropout Learning

Dropout learning adalah teknik *regularisasi* untuk meredam *overfitting* pada jaringan saraf, termasuk *CNN*. Mekanismenya menonaktifkan sebagian unit/neuron secara acak (sekitar 50%) di setiap iterasi pelatihan, agar jaringan tidak bergantung pada neuron tertentu dan mendorong tiap neuron mempelajari fitur yang lebih umum dan tangguh. Strategi ini meningkatkan kemampuan generalisasi terhadap data yang belum pernah dilihat serta mengurangi *co-adaptation* antar neuron, sehingga kinerja keseluruhan membaik[10].

2.2.7 Confusion Matrix

Confusion matrix adalah tabel evaluasi untuk menilai kinerja model klasifikasi, yang menunjukkan jumlah prediksi benar dan salah dengan membandingkan hasil prediksi terhadap label sebenarnya. Dalam pengenalan pola, *confusion matrix* membantu menilai kemampuan model membedakan antar kelas serta mengidentifikasi pola kesalahan yang terjadi[26]. *Confusion matrix* mempunyai elemen-elemen berikut:

- *True Positive (TP)* : jumlah kasus positif yang diprediksi benar.
- *True Negative (TN)* : jumlah kasus negatif yang diprediksi benar.
- *False Positive (FP)* : jumlah kasus yang diprediksi positif padahal sebenarnya negatif.
- *False Negative (FN)* : jumlah kasus yang diprediksi negatif padahal sebenarnya positif.

Dari elemen-elemen ini, dapat dihitung metrik evaluasi seperti *akurasi*, *presisi*, *recall*, dan *F1-score* untuk menilai kinerja model secara menyeluruh. Tabel 2.2

menyajikan struktur *confusion matrix* secara umum[27].

Tabel 2.2 Confusion Matrix

		AKTUAL	
		Positif	**Negatif**
PREDIKSI	**Positif**	*True Positive (TP)*	*False Positive (FP)*
	Negatif	*False Negative (FN)*	*True Negative (TN)*

Pada kasus multiclass dengan 5 kelas, *confusion matrix* berbentuk matriks 5×5 yang menampilkan jumlah prediksi per pasangan kelas aktual–prediksi. Tabel 2.3 menyajikan struktur *multiclass confusion matrix* secara umum[27].

Tabel 2.3 Multiclass Confusion Matrix (5 Kelas)

		AKTUAL				
		Kelas 1	**Kelas 2**	**Kelas 3**	**Kelas 4**	**Kelas 5**
PREDIKSI	**Kelas 1**	TP_1	$FP_{1,2}$	$FP_{1,3}$	$FP_{1,4}$	$FP_{1,5}$
	Kelas 2	$FP_{2,1}$	TP_2	$FP_{2,3}$	$FP_{2,4}$	$FP_{2,5}$
	Kelas 3	$FP_{3,1}$	$FP_{3,2}$	TP_3	$FP_{3,4}$	$FP_{3,5}$
	Kelas 4	$FP_{4,1}$	$FP_{4,2}$	$FP_{4,3}$	TP_4	$FP_{4,5}$
	Kelas 5	$FP_{5,1}$	$FP_{5,2}$	$FP_{5,3}$	$FP_{5,4}$	TP_5

2.2.7.1 Accuracy

Accuracy merupakan metrik evaluasi yang paling lazim digunakan untuk menilai kinerja model klasifikasi. Meskipun intuitif, metrik ini berpotensi menyesatkan pada kondisi ketidakseimbangan kelas yang mencolok, karena model dapat meraih nilai akurasi tinggi hanya dengan memprediksi kelas mayoritas[28]. Perhitungan *accuracy* disusun berdasarkan komponen pada *confusion matrix* sebagai berikut:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{Rumus 2.4}$$

2.2.7.2 Precision

Precision mengukur proporsi prediksi positif yang benar (TP) terhadap seluruh prediksi positif yang dibuat model (TP + FP)[28]. Metrik ini menilai ketepatan prediksi positif dan menjadi krusial ketika kesalahan positif (*false positive*) perlu diminimalkan. Perhitungan *precision* berdasarkan elemen pada *confusion matrix* adalah:

$$\text{Precision} = \frac{TP}{TP + FP}$$

(Rumus 2.5)

2.2.7.3 Recall

Recall (sensitivitas) mengukur proporsi kasus positif yang teridentifikasi benar oleh model dibandingkan dengan seluruh kasus positif yang sebenarnya. Metrik ini merefleksikan kemampuan model menangkap sebanyak mungkin kasus positif dan menjadi krusial ketika kesalahan negatif (*false negative*) perlu diminimalkan[28]. Perhitungan *recall* berdasarkan elemen pada *confusion matrix* adalah:

$$\text{Recall} = \frac{TP}{TP + FN}$$

(Rumus 2.6)

2.2.7.4 F1-Score

F1-score merupakan metrik kinerja klasifikasi yang didefinisikan sebagai rata-rata harmonik antara *precision* dan *recall*. Metrik ini banyak digunakan pada temu kembali informasi dan evaluasi klasifikasi, khususnya pada skenario dengan prevalensi kelas rendah, karena mampu menyeimbangkan keduanya [29]. Dengan demikian, *F1-score* memberikan ringkasan kinerja yang lebih komprehensif dibandingkan hanya mengandalkan *precision* atau *recall*. Perhitungan *F1-score* dari *precision* dan *recall* adalah:

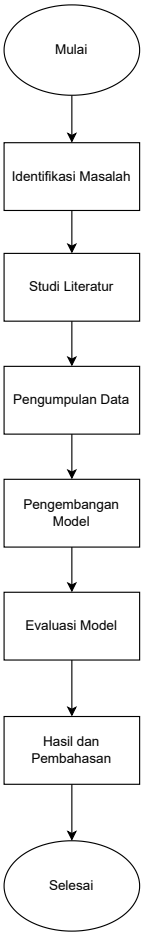
$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

(Rumus 2.7)

BAB III
METODE PENELITIAN

3.1 Alur Penelitian

Alur penelitian tugas akhir ini—klasifikasi suara lingkungan berbasis *Piczak-CNN* dengan masukan spektrogram—ditunjukkan pada Gambar 3.1.



Gambar 3.1 Alur Penelitian

3.2 Penjabaran Langkah Penelitian

Mengacu pada alur pada Gambar 3.1, berikut uraian ringkas langkah-langkah penelitian yang dilaksanakan.

3.2.1 Identifikasi Masalah

Penelitian ini berangkat dari kebutuhan membantu penyandang gangguan pendengaran mengenali suara lingkungan saat bepergian. Ketidakmampuan mendeteksi suara peringatan (mis. klakson, sirene, pengumuman) menurunkan keselamatan dan kenyamanan mobilitas. Permasalahan utamanya adalah belum tersedianya sistem pengenal suara lingkungan yang andal, akurat, dan mudah diakses pada perangkat bergerak. Karena itu, dirancang sistem klasifikasi suara lingkungan berbasis representasi spektrogram dengan arsitektur *Piczak-CNN* untuk meningkatkan aksesibilitas bagi pengguna dengan keterbatasan pendengaran.

3.2.2 Studi Literatur

Dilakukan kajian pustaka terhadap artikel ilmiah, jurnal, buku, dan sumber tepercaya terkait pengenalan suara lingkungan, representasi spektrogram, dan arsitektur *Piczak-CNN*. Tujuannya memahami konsep dasar, metode yang ada, serta tantangan pengembangan sistem klasifikasi suara lingkungan, sebagai landasan teoretis penelitian.

3.2.3 Pengumpulan Data

Data audio dikumpulkan dari sumber daring legal dan berlisensi untuk penelitian, serta perekaman langsung di lapangan. Untuk menyerupai skenario penggunaan nyata, seluruh cuplikan audio dari internet direkam ulang menggunakan mikrofon *smartphone* agar karakteristik akustiknya mendekati kondisi pengguna saat bergerak, kondisi lingkungan saat perekaman ulang dilakukan dalam kamar biasa dengan tingkat kebisingan latar rendah dengan cara audio digital diputar melalui pengeras suara dan direkam kembali

menggunakan *smartphone*. Setiap sampel audio dikonversi ke format *WAV*, mono, laju sampel 44,100 Hz, kedalaman bit 16 bit, durasi 4 detik.

Pemilihan lima kategori suara (klakson mobil, klakson motor, sirene, kaca pecah, tangisan bayi) didasarkan pada urgensi keselamatan dan peningkatan kesadaran situasional bagi penyandang gangguan pendengaran. Tiga kelas pertama merupakan sinyal peringatan lalu lintas kritis yang banyak digunakan pada sistem peringatan berbasis audio untuk pengguna tunarungu [2], [3], [4], [5], [6], [7]. Suara kaca pecah menjadi indikator insiden atau keadaan darurat pada konteks keamanan lingkungan [13], [14], [15]. Tangisan bayi relevan dengan respon cepat dalam konteks keselamatan dan kesejahteraan anak pada lingkungan domestik [13], [14], [16]. Kombinasi ini membentuk dasar sistem peringatan dini yang aplikatif di lingkungan publik maupun domestik.

3.2.4 Pengembangan Model

Model klasifikasi suara lingkungan dikembangkan dengan arsitektur *Piczak-CNN* pada *PyTorch*. Jaringan terdiri atas lapisan *convolution*, *pooling*, dan *fully connected* dan menerima spektrogram *Mel* hasil pra-pemrosesan dan menghasilkan prediksi label kelas. Pra-pemrosesan meliputi normalisasi amplitudo, ekstraksi fitur spektrogram *Mel*, dan *augmentation* data untuk meningkatkan keragaman sampel latih. Pada konfigurasi *baseline*, pelatihan menggunakan *optimizer Stochastic Gradient Descent* (*SGD*) dengan *learning rate* tetap, serta regularisasi *dropout* untuk mencegah *overfitting*; *Adam* digunakan sebagai model pembanding pada studi ablasi.

3.2.5 Evaluasi Model

Evaluasi model dilakukan untuk menilai kemampuan sistem dalam mengklasifikasikan suara lingkungan pada lima kelas. Pada data latih diterapkan validasi silang *k-fold* (k=5) untuk pemilihan hiperparameter dan estimasi variansi performa. Kinerja dievaluasi menggunakan *accuracy*,

precision, *recall*, F1-*score*, serta *confusion matrix*. Selain itu, dilakukan studi ablasi untuk menilai kontribusi tiap komponen pra-pemrosesan terhadap hasil akhir.

3.2.6 Hasil dan Pembahasan

Kinerja dilaporkan dengan *accuracy*, *precision*, *recall*, F1-*score*, serta *confusion matrix* untuk distribusi prediksi antarkelas. Temuan dianalisis untuk menilai kemampuan model, kelebihan, keterbatasan, dan perbandingan dengan studi terdahulu.

3.3 Alat dan Bahan Tugas Akhir

Bagian ini menguraikan alat dan bahan yang digunakan dalam penelitian.

3.3.1 Alat

Alat yang digunakan:

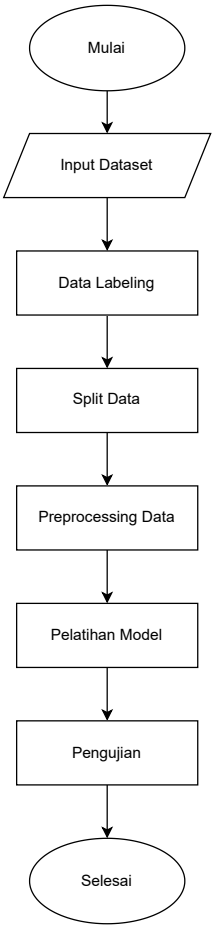
1. Laptop: Intel(R) Core(TM) i5-10300H @ 2,50 GHz (8 *CPU*), RAM 8 GB, *GPU* Nvidia GeForce GTX 1650.
2. *Smartphone*: *OS Android* 1.0.6.0, *CPU Snapdragon* 685 *Octa-core*, RAM 6 GB, penyimpanan 128 GB.
3. Speaker JBL GO2.
4. *Visual Studio Code* sebagai *text editor*.
5. Python versi 3.13.6

3.3.2 Bahan

Bahan penelitian berupa data audio suara lingkungan lima kategori yaitu klakson mobil, klakson motor, sirene, kaca pecah, dan tangisan bayi. Data diperoleh dari sumber daring berlisensi serta rekaman lapangan menggunakan *smartphone*. Total audio sebanyak 500 cuplikan berdurasi 4 detik per sampel dalam format WAV.

3.4 Metode Pengembangan Model

Metode pengembangan model pada penelitian ini mengacu pada arsitektur *Piczak-CNN* untuk klasifikasi suara lingkungan. Model ini dirancang untuk mengekstraksi fitur dari spektrogram *Mel* dan mengklasifikasikannya ke dalam lima kategori suara. Alur pengembangan model ditunjukkan pada Gambar 3.2.



Gambar 3.2 Metode Pengembangan Model

3.4.1 *Input Dataset*

Tahap awal penelitian mencakup penyusunan *dataset* audio suara lingkungan yang diperoleh dari sumber daring berlisensi serta rekaman lapangan. *Dataset* mencakup lima kategori suara yaitu klakson mobil, klakson motor, sirene, kaca pecah, dan tangisan bayi dengan total 500 cuplikan audio berdurasi 4 detik per sampel dalam format WAV.

3.4.2 *Data Labeling*

Agar model dapat belajar mengklasifikasikan suara, setiap sampel audio diberi label sesuai kategorinya yaitu sirine (label 0), klakson mobil (label 1), klakson motor (label 2), kaca pecah (label 3), dan tangisan bayi (label 4). Penetapan label ini penting untuk memastikan data latih memiliki anotasi yang akurat. Dengan adanya label ini, model dapat memetakan fitur audio ke kelas yang benar selama pelatihan. Proses *data labeling* ditunjukkan pada Tabel 3.1.

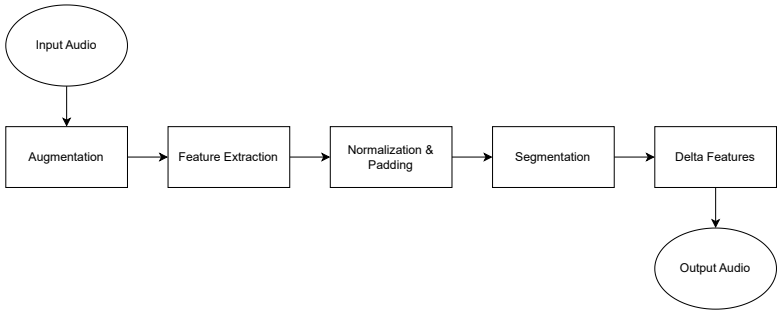
Tabel 3.1 Data Labeling

Kelas Suara	**Label**
Sirene	0
Klakson Mobil	1
Klakson Motor	2
Kaca pecah	3
Tangisan Bayi	4

3.4.3 *Split* Data

Pembagian data menggunakan skema *5-fold cross-validation*. Seluruh sampel diacak (*shuffled*) dengan *seed* tetap (42) untuk menjamin reprodusibilitas. Dataset dibagi ke dalam lima lipatan, dan pada setiap iterasi dialokasikan sebagai himpunan latih dan sebagai himpunan validasi. Skema pembagian diserialisasi dalam berkas JSON guna menjaga konsistensi antar skenario pengujian. Setiap lipatan disusun secara *stratified* agar proporsi kelas tetap seimbang selama pelatihan dan evaluasi.

3.4.4 *Pre-processing* Data



Gambar 3.3 Alur *Pre-processing* Data

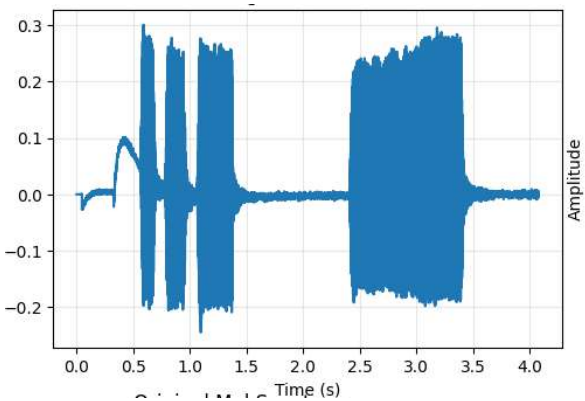
3.4.4.1 *Augmentation*

Untuk meningkatkan keragaman data latih dan mencegah *overfitting*, dilakukan *augmentation* data dengan teknik berikut. Augmentasi hanya diterapkan pada data latih (*training set*) pada setiap *epoch*, sedangkan data validasi dibiarkan tanpa augmentasi agar evaluasi mencerminkan performa model pada data asli.

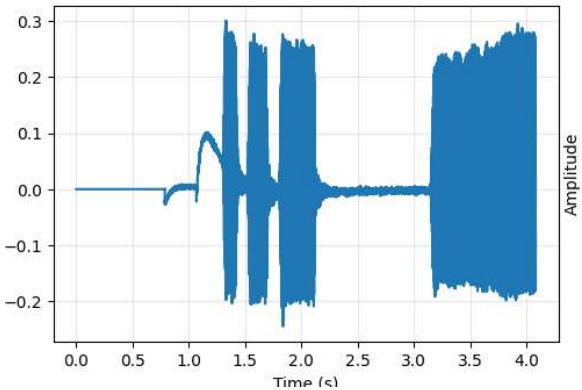
Alur penerapan augmentasi pada setiap sampel latih adalah: (1) sinyal *waveform* dimuat, (2) *time shifting* opsional, (3) penambahan derau opsional, (4) ekstraksi *Mel-spectrogram*, lalu (5) *spectrogram augmentation* opsional. Urutan ini dipilih agar augmentasi pada domain waktu dilakukan sebelum ekstraksi fitur, sementara augmentasi domain spektrogram dilakukan setelah fitur terbentuk.

Rincian teknik yang digunakan adalah sebagai berikut:

1. *Time Shifting*: Menggeser sinyal audio secara acak ke kiri atau kanan dengan jarak hingga 20% dari durasi total sinyal dan mengisi bagian kosong dengan nol. Tujuan teknik ini adalah untuk mensimulasikan variasi waktu kedatangan suara di lingkungan nyata. Berikut ilustrasi *time shifting* pada Gambar 3.4.

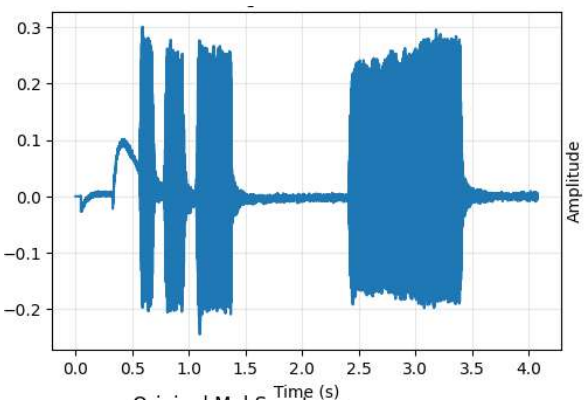


Gambar 3.4 Sebelum *Time Shifting*

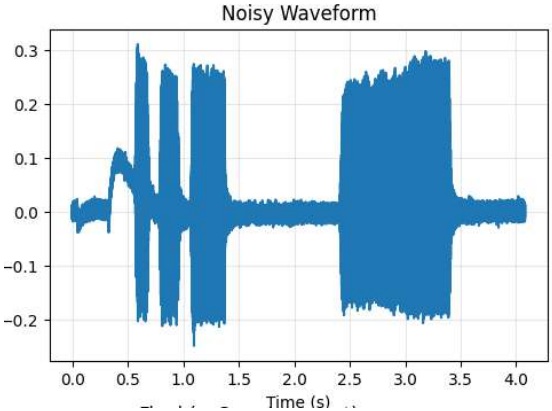


Gambar 3.5 Sesudah *Time Shifting*

2. *Adding Background Noise*: Derau Gaussian putih ditambahkan secara aditif ke sinyal audio untuk mensimulasikan kondisi lingkungan yang bising. Derau dibangkitkan dari distribusi normal standar ($\mu = 0, \sigma = 1$). Teknik ini digunakan untuk meningkatkan ketahanan model terhadap variasi kebisingan pada skenario nyata. Berikut ilustrasi penambahan derau pada Gambar 3.6.

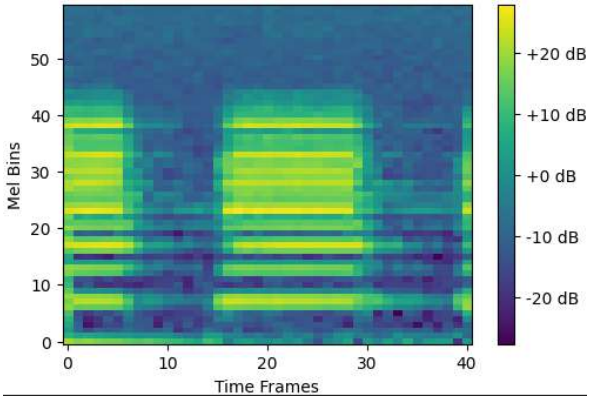


Gambar 3.6 Sebelum Penambahan Derau

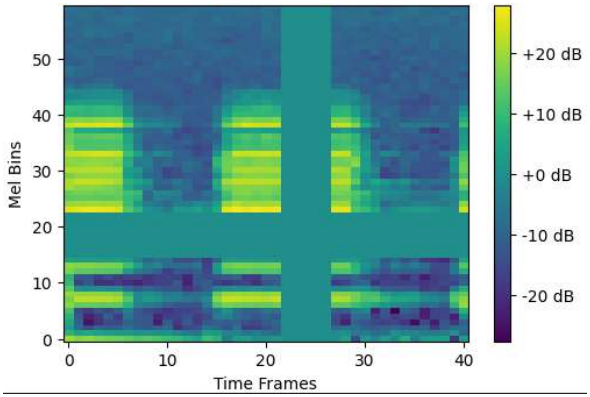


Gambar 3.7 Sesudah Penambahan Derau

3. *Spectrogram Augmentation*: Menerapkan *augmentation* pada domain spektrogram dengan menyembunyikan frekuensi acak (hingga 8 *bin*) dan waktu acak (hingga 10 *frame*). Teknik ini membantu model menjadi lebih tangguh terhadap variasi spektral dan temporal. Augmentasi ini dilakukan setelah ekstraksi fitur spektrogram *Mel*. Berikut ilustrasi *spectrogram augmentation* pada Gambar 3.8.



Gambar 3.8 Sebelum *Spectrogram Augmentation*



Gambar 3.9 Sesudah *Spectrogram Augmentation*

Setiap teknik *augmentation* diterapkan secara acak pada setiap sampel audio selama pelatihan dengan probabilitas 50% per teknik (independen). Dengan demikian, dalam satu iterasi sebuah sampel dapat menerima nol, satu, dua, atau tiga augmentasi sekaligus sesuai hasil pengacakan.

Skenario penerapan augmentasi pada penelitian ini dijelaskan sebagai berikut:

1. Skenario A (Baseline - Augmentasi Aktif): ketiga teknik augmentasi

aktif dengan probabilitas masing-masing 50% per sampel pada setiap *epoch*. Ini menghasilkan variasi data latih yang dinamis antarepoch.

2. Skenario B (Ablasi - Augmentasi Nonaktif): seluruh augmentasi dinonaktifkan, sehingga model dilatih hanya dengan data asli. Skenario ini digunakan untuk mengukur kontribusi augmentasi terhadap peningkatan performa.

Formulasi peluang untuk setiap teknik dapat ditulis sebagai $P(\text{augmentasi aktif}) = 0{,}5$ dan $P(\text{augmentasi nonaktif}) = 0{,}5$. Karena ketiga teknik bersifat independen, peluang tidak ada augmentasi yang aktif pada satu sampel adalah $(0{,}5)^3 = 0{,}125$ (12,5%), sedangkan peluang minimal satu augmentasi aktif adalah $1 - 0{,}125 = 0{,}875$ (87,5%).

3.4.4.2 *Feature Extraction*

Setelah *augmentation*, setiap sampel audio diubah menjadi spektrogram *Mel* dan dB skala log. Proses ini melibatkan:

1. Menghitung spektrogram *STFT* dengan jendela *Hamming* berukuran 1024 sampel (23 ms pada 44,100 Hz), tumpang tindih 50% (512 sampel) dan 60 *Mel bin*.
2. Mengonversi amplitudo ke skala dB logaritmik untuk menyesuaikan persepsi pendengaran manusia.
3. Menghitung fitur *delta* terhadap waktu dari spektrogram *Mel* untuk menangkap perubahan temporal.

Hasil akhirnya adalah *tensor* berukuran $1 \times 60 \times 259$ ($C \times F \times T$) per sampel, di mana C=1 (kanal tunggal), F=60 (*Mel bin*), dan T=259 (*frame* waktu).

3.4.4.3 *Normalization* dan *Padding*

Jika jumlah *frame* waktu kurang dari 41, dilakukan *padding* dengan menambahkan *frame* nol di akhir hingga mencapai panjang 41 *frame*. Normalisasi dan *padding* ini memastikan konsistensi dimensi masukan ke

model.

3.4.4.4 *Segmentation*

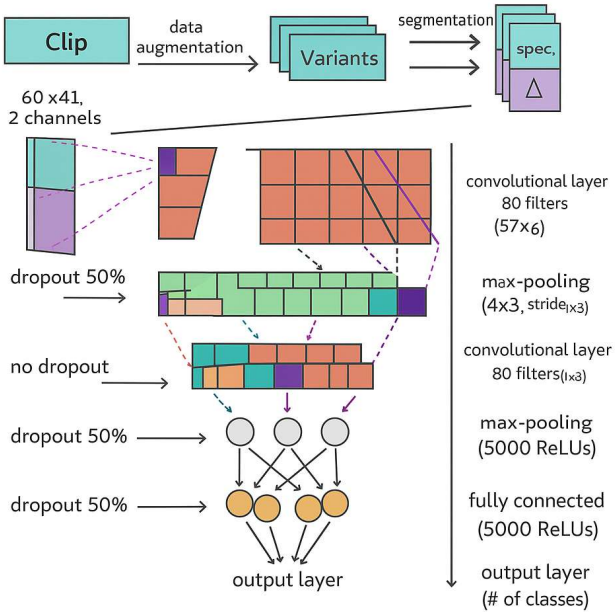
Setelah normalisasi dan *padding*, setiap spektrogram *Mel* diambil potongan acak sepanjang 41 *frame* waktu dengan durasi kira-kira 0,47 detik (41 *frame* × 10 ms/*frame*). Teknik ini membantu model belajar dari berbagai segmen sinyal audio, meningkatkan generalisasi terhadap variasi temporal dalam data nyata.

3.4.4.5 *Delta Features*

Fitur *delta* terhadap waktu dihitung dari spektrogram *Mel* untuk menangkap perubahan temporal. Fitur *delta* ini ditumpuk bersama spektrogram *Mel* asli pada dimensi kanal, menghasilkan *tensor* berukuran 2×60×41 (C × F × T) per sampel. Kanal pertama berisi spektrogram *Mel* dalam skala log sebagai fitur statis, sedangkan kanal kedua berisi fitur *delta* sebagai fitur dinamis.

3.4.5 Pelatihan Model

Metode yang digunakan adalah *Piczak-CNN*, terdiri atas lapisan *convolution*, *pooling*, dan *fully connected* untuk mengekstraksi fitur dari spektrogram *Mel*. Arsitektur yang digunakan ditunjukkan pada Gambar 3.10.



Gambar 3.10 Arsitektur *Piczak-CNN* [10]

3.4.6 Lapisan Input

Masukan berbentuk *tensor* berukuran $2 \times 60 \times 41$ (C × F × T). Kanal pertama berisi spektrogram *Mel* dalam skala log sebagai fitur statis, kanal kedua berisi fitur *delta* terhadap waktu sebagai fitur dinamis dan keduanya ditumpuk pada dimensi kanal. Dimensi frekuensi memiliki 60 *Mel bin*, sedangkan dimensi waktu 41 *frame*.

3.4.6.1 Lapisan Konvolusi Pertama (Conv1)

Tensor $2 \times 60 \times 41$ dikonvolusikan dengan 80 *kernel* berukuran 57×6 (*stride* 1, tanpa *padding*), sesuai spesifikasi *Piczak-CNN*. Cakupan *kernel* hampir penuh pada sumbu frekuensi (57 dari 60 *Mel bin*) dengan rentang temporal pendek (6 *frame*) untuk menangkap keterkaitan spektral temporal lokal. Keluaran berukuran $80 \times 4 \times 36$, diikuti aktivasi *ReLU* dan kemudian *max pooling* 4×3.

3.4.6.2 Lapisan Max Pooling Pertama (MaxPool1)

Setelah *ReLU*, peta fitur $80 \times 4 \times 36$ diproses dengan *max pooling* berukuran 4×3 (*stride* 4×3, tanpa *padding*). Operasi ini merangkum informasi pada 4 *bin* frekuensi dan 3 *frame* waktu, menghasilkan $80 \times 1 \times 12$, sejalan dengan rancangan *Piczak-CNN*.

3.4.6.3 Lapisan Max Pooling Kedua (MaxPool2)

Setelah aktivasi pada keluaran *Conv2*, peta fitur $80 \times 1 \times 10$ diperkecil menggunakan *max pooling* 1×3 (*stride* 1×3, tanpa *padding*) menjadi $80 \times 1 \times 3$. Representasi ini kemudian diratakan sebelum masuk ke lapisan *fully connected*.

3.4.6.4 Lapisan Fully Connected dan Regularisasi

Keluaran $80 \times 1 \times 3$ diratakan menjadi vektor 240 elemen dan diproses dua lapis *fully connected* berukuran 5000 unit dengan aktivasi *ReLU*. *Dropout* 50% diterapkan sebelum dan di antara keduanya untuk menekan *overfitting*. Lapisan keluaran *linear* memiliki 5 unit sesuai jumlah kelas dan diikuti *Softmax* saat inferensi. Alur dimensi: $240 \rightarrow 5000 \rightarrow 5000 \rightarrow 5$.

3.4.6.5 Lapisan Output

Lapisan output berupa *linear* dengan lima unit (klakson mobil, klakson motor, sirene, knalpot motor, mesin mobil), menghasilkan *logit* per kelas yang dikonversi menjadi probabilitas melalui *Softmax* saat inferensi. Selama

pelatihan digunakan *cross-entropy loss* (menggabungkan *Softmax* secara implisit).

3.4.7 *Hyperparameter* Tuning

Setelah penetapan arsitektur model, dilakukan optimisasi terhadap *hyperparameter* untuk meningkatkan performa klasifikasi. Optimisasi mencakup pemilihan algoritma *optimizer*, tingkat pembelajaran (*learning rate*), koefisien regularisasi *dropout*, dan strategi augmentasi data. Rincian nilai *hyperparameter* utama(*baseline*) yang digunakan dalam penelitian ini disajikan pada Tabel 3.2.

Tabel 3.2 *Hyperparameter* Default Model

Hyperparameter	**Nilai**	**Deskripsi**
Optimizer	SGD	Stabil pada *dataset* menengah-kecil dan mudah dikontrol saat evaluasi ablation
Learning Rate	2e-3	Nilai moderat untuk menjaga keseimbangan kecepatan konvergensi dan kestabilan pelatihan
Dropout Rate	0,5	Regularisasi kuat pada lapisan *fully connected* berukuran besar agar *overfitting* berkurang
Time Shifting	Aktif	Menambah variasi posisi temporal kejadian suara tanpa mengubah label kelas
Background Noise	Aktif	Meningkatkan ketahanan model terhadap kondisi kebisingan nyata
Spectrogram Augmentation	Aktif	Mendorong model belajar pola global dengan menutupi area waktu/frekuensi secara acak
Jumlah *Epoch*	50	Cukup untuk konvergensi pada data 500 sampel tanpa biaya komputasi berlebihan
Ukuran *Batch*	32	Kompromi efisiensi memori GPU dan kestabilan estimasi gradien

Tabel 3.2 menyajikan nilai *hyperparameter* default yang digunakan dalam pelatihan model. Nilai-nilai tersebut ditetapkan berdasarkan kajian literatur, karakteristik arsitektur *Piczak-CNN*, ukuran *dataset* (500 sampel), dan keterbatasan komputasi perangkat yang digunakan. Penyesuaian lebih lanjut dilakukan secara iteratif melalui evaluasi pada data validasi untuk mencapai performa optimal.

Alasan pemilihan *hyperparameter* utama pada konfigurasi *baseline* diringkas sebagai berikut:

1. *Optimizer* SGD dipilih sebagai *baseline* karena proses pembaruan bobot yang lebih konservatif cenderung memberikan generalisasi baik pada *dataset* berukuran terbatas; *Adam* tetap diuji pada ablasi untuk membandingkan kecepatan konvergensi.
2. *Learning rate* 2e-3 dipilih sebagai nilai tengah agar pelatihan tidak terlalu lambat (seperti laju terlalu kecil) dan tidak mudah berosilasi (seperti laju terlalu besar).
3. *Dropout rate* 0,5 digunakan karena lapisan *fully connected* berukuran 5000 unit memiliki kapasitas tinggi, sehingga regularisasi lebih kuat diperlukan untuk menekan *overfitting*.
4. Tiga teknik augmentasi (*time shifting*, *background noise*, dan *spectrogram augmentation*) diaktifkan untuk meningkatkan keragaman data latih dan ketahanan model terhadap variasi temporal, kebisingan, serta gangguan spektral.
5. Jumlah *epoch* 50 dipilih agar model memiliki kesempatan konvergensi yang memadai pada skema 5-*fold* tanpa memperpanjang waktu komputasi secara berlebihan.
6. Ukuran *batch* 32 dipilih sebagai kompromi antara pemanfaatan memori GPU, stabilitas gradien, dan efisiensi waktu pelatihan.

Untuk menilai kontribusi individual setiap komponen terhadap performa model, dilakukan studi ablasi dengan variasi sistematis pada *hyperparameter*

kunci. Studi ablasi berfokus pada pengaruh koefisien *dropout*, teknik *augmentation* data, serta pemilihan algoritma *optimizer* terhadap metrik evaluasi yang meliputi *accuracy*, *precision*, *recall*, dan F1-*score*. Hasil studi ablasi akan dianalisis untuk mengidentifikasi komponen yang paling berpengaruh dalam meningkatkan kinerja klasifikasi suara lingkungan. Detail skenario pengujian studi ablasi disajikan pada Tabel 3.3.

Tabel 3.3 *Hyperparameter* Studi Ablasi

No	Hyperparameter	Variasi	Keterangan
1	*Optimizer*	SGD, Adam	Perbandingan algoritma optimasi pembaruan bobot
2	*Learning Rate*	1e-3, 2e-3	Pengaruh tingkat perubahan bobot per iterasi
3	*Dropout Rate*	0,0 ; 0,1 ; 0,2 ; 0,3 ; 0,5	Pengaruh koefisien dropout pada regularisasi
4	*Augmentation*	Aktif, Nonaktif	Evaluasi kontribusi teknik augmentasi data
-	Jumlah *Epoch*	50	Dampak jumlah iterasi pelatihan pada performa
-	Ukuran *Batch*	32	Pengaruh jumlah sampel per pembaruan bobot

Tabel 3.3 menyajikan rancangan sistematis variasi *hyperparameter* yang akan diuji melalui studi ablasi. Setiap skenario pengujian dievaluasi menggunakan metrik yang konsisten untuk memastikan perbandingan yang objektif dan dapat dipertanggungjawabkan. Melalui analisis menyeluruh terhadap hasil studi ablasi ini, dapat diidentifikasi komponen-komponen kritikal yang memberikan kontribusi signifikan terhadap peningkatan performa model klasifikasi suara lingkungan.

3.5 Ilustrasi Perhitungan

Pada subbab ini disajikan contoh perhitungan metrik evaluasi berbasis *confusion matrix* untuk kasus klasifikasi biner dengan jumlah sampel $N = 100$. Data bersifat hipotetik dan digunakan semata untuk mengilustrasikan cara memperoleh *accuracy*, *precision*, *recall*, *specificity*, dan F1-*score*. Matriks

kebingungan dengan ukuran 5×5 disajikan pada Tabel 3.4.

Tabel 3.4 Contoh *Confusion Matrix* dengan Struktur Header

		AKTUAL				
		Kelas 1	**Kelas 2**	**Kelas 3**	**Kelas 4**	**Kelas 5**
PREDIKSI	**Kelas 1**	18	2	1	0	0
	Kelas 2	3	15	2	1	0
	Kelas 3	0	1	16	2	1
	Kelas 4	0	0	2	14	3
	Kelas 5	0	0	0	3	16

Berdasarkan Tabel 3.4, perhitungan metrik evaluasi untuk Kelas 1 adalah sebagai berikut:

- *True Positive (TP)*: 18
- *True Negative (TN)*: 76
- *False Positive (FP)*: 3
- *False Negative (FN)*: 3

Dengan nilai TP, TN, FP, dan FN tersebut, metrik evaluasi kelas 1 dihitung sebagai berikut:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{18 + 76}{18 + 76 + 3 + 3} = \frac{94}{100} = 0,94 \text{ atau } 94\% \tag{Rumus 3.1}$$

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{18}{18 + 3} = \frac{18}{21} \approx 0,857 \text{ atau } 85,7\% \tag{Rumus 3.2}$$

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{18}{18 + 3} = \frac{18}{21} \approx 0,857 \text{ atau } 85,7\% \tag{Rumus 3.3}$$

$$F1\text{-}score = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = 2 \times \frac{0,857 \times 0,857}{0,857 + 0,857} \approx 0,857 \text{ atau } 85,7\% \tag{Rumus 3.4}$$

3.6 Rencana Pengujian

Rencana pengujian ini dirancang untuk mengevaluasi kinerja model klasifikasi suara lingkungan berbasis *Piczak-CNN*. Prosedur pengujian dilaksanakan secara sistematis guna memastikan hasil evaluasi yang andal, terukur, dan dapat direplikasi. Tahapan pengujian yang diusulkan diuraikan pada bagian berikut.

3.6.1 Studi Ablasi

3.6.1.1 Konsep Studi Ablasi

Studi ablasi adalah metode evaluasi yang melibatkan penghapusan atau modifikasi komponen tertentu dari model atau proses untuk menilai kontribusi masing-masing terhadap kinerja keseluruhan. Dalam konteks penelitian ini, studi ablasi akan dilakukan dengan menghilangkan atau mengubah elemen-elemen kunci dari *pipeline* pra-pemrosesan data dan arsitektur *Piczak-CNN*. Tujuannya adalah untuk memahami dampak setiap komponen terhadap *accuracy* klasifikasi suara lingkungan.

3.6.1.2 Skenario Pengujian

Penelitian ini merancang empat skenario pengujian untuk studi ablasi, yaitu:

1. Skenario 1 - *Baseline* (Model Lengkap)

   Pada skenario pertama ini digunakan konfigurasi dasar (*baseline*) dengan arsitektur *Piczak-CNN* lengkap sebagaimana dirancang. Seluruh komponen aktif yaitu *dropout* pada lapisan *fully connected* dan tiga teknik *augmentation* (*time shifting*, penambahan derau *Gaussian*, dan *spectrogram augmentation*/*SpecAugment*) yang diterapkan secara berurutan dengan probabilitas masing-masing 50%. *Optimizer* yang digunakan adalah *Stochastic Gradient Descent (SGD)* dengan parameter *learning rate*, *momentum* dan *Nesterov* diaktifkan. Pelatihan dilakukan

selama 50 *epoch* dengan ukuran *batch* 32. Evaluasi kinerja dilaksanakan menggunakan 5-*fold cross-validation* untuk memperoleh *accuracy* validasi terbaik tiap *fold*. Hasil skenario ini menjadi acuan utama untuk membandingkan pengaruh penghilangan komponen pada skenario ablasi berikut.

2. Skenario 2 - Eksperimentasi *Dropout*

   Pada skenario kedua ini dilakukan variasi *dropout rate* (0,0; 0,1; 0,2; 0,3) dengan komponen lain sama seperti *baseline* (*optimizer* SGD, *learning rate* 0,002, augmentasi aktif). Tujuannya adalah mengidentifikasi nilai *dropout* yang paling sesuai untuk karakteristik data penelitian.

3. Skenario 3 - Eksperimentasi *Augmentation*

   Pada skenario ketiga ini dilakukan perbandingan augmentasi aktif dan nonaktif dengan menggunakan nilai *dropout* terbaik dari skenario 2, sementara *optimizer* dan *learning rate* tetap. Skenario ini digunakan untuk menilai kontribusi nyata augmentasi terhadap performa klasifikasi pada data penelitian.

4. Skenario 4 - Eksperimentasi *Optimizer*

   Pada skenario keempat ini dilakukan perbandingan *optimizer* SGD dan Adam menggunakan konfigurasi terbaik dari skenario 3 agar pengaruh algoritma optimisasi dapat diukur secara adil. Pelatihan dan evaluasi tetap menggunakan prosedur 5-*fold cross-validation* yang sama.

BAB IV
HASIL DAN PEMBAHASAN

4.1 Hasil

Bab ini menyajikan hasil penelitian dan pengujian yang telah dilakukan berdasarkan rancangan yang sudah dijelaskan pada Bab III. Pada penelitian ini, tahapan yang dilaksanakan meliputi pengumpulan data, *pre-processing* dan augmentasi data, pengembangan model, serta evaluasi model. Setiap tahapan dijelaskan secara rinci pada subbab berikut.

4.1.1 Pengumpulan Data

Dataset yang digunakan pada penelitian ini berasal dari repositori sumber terbuka, seperti Freesound.org dan ESC-50 (https://github.com/karolpiczak/ESC-50.git), serta dari perekaman audio mandiri. Seluruh berkas audio yang diperoleh dari internet direkam ulang menggunakan mikrofon pada telepon pintar untuk menjaga konsistensi proses akuisisi. Dataset mencakup lima kelas suara yaitu klakson mobil, klakson motor, sirene, pecahan kaca, dan tangisan bayi. Setiap kelas berisi 100 sampel sehingga total keseluruhan berjumlah 500 sampel audio. Setiap sampel berformat WAV dengan laju sampel 44,1 kHz dan saluran tunggal (mono).

4.1.2 Split Data

Data yang telah dikumpulkan dibagi menggunakan 5-fold cross-validation untuk memastikan evaluasi yang adil dan komprehensif terhadap model. Pembagian data dilakukan sedemikian rupa sehingga setiap fold memiliki proporsi yang seimbang dari setiap kelas suara. Setiap fold terdiri dari data untuk pelatihan dan untuk validasi. Proses pembagian data ini diatur dalam berkas konfigurasi `split.json` yang memuat informasi mengenai indeks

sampel yang termasuk dalam setiap fold.

Kode 4.1 Implementasi kode 5-fold-cross-validation

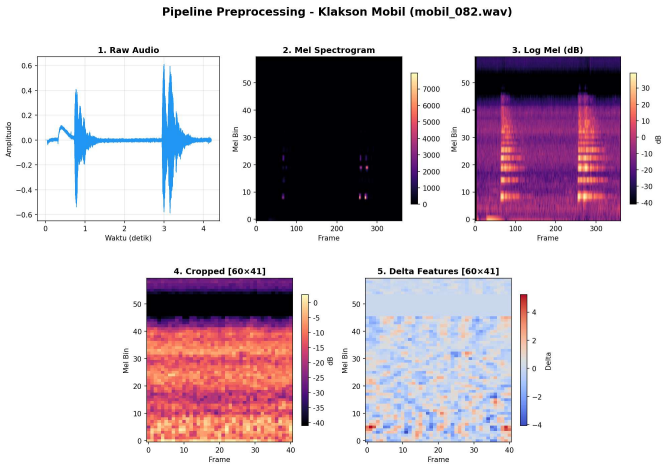
```
import os
import random
import json

AUDIO_EXTS = ('.wav',)

def make_5fold_split(data_dir=None, n_folds=5, seed=42):
        if data_dir is None:
                data_dir = os.path.join(os.getcwd(), 'data')

        samples = []
        for cls in sorted(os.listdir(data_dir)):
                cls_dir = os.path.join(data_dir, cls)
                if not os.path.isdir(cls_dir):
                        continue
                for root, _, files in os.walk(cls_dir):
                        for f in files:
                                if f.lower().endswith(AUDIO_EXTS):
                                        abs_path = os.path.abspath(os.path.join(root, f))
                                        samples.append((abs_path, cls))

        if not samples:
                raise RuntimeError(f"Tidak menemukan file audio di {data_dir}")

        random.seed(seed)
        random.shuffle(samples)

        folds = [[] for _ in range(n_folds)]
        for i, s in enumerate(samples):
                folds[i % n_folds].append(s)

        split = []
        for i in range(n_folds):
                val = folds[i]
                train = [s for j, fj in enumerate(folds) if j != i for s in fj]
                split.append({"train": train, "val": val})

        return split

if __name__ == "__main__":
        data_dir = os.path.join(os.getcwd(), 'data')
        split = make_5fold_split(data_dir, n_folds=5, seed=42)
        out = os.path.join(os.getcwd(), "split.json")
        with open(out, "w", encoding="utf-8") as f:
                json.dump(split, f, indent=2, ensure_ascii=False)
        print(f"Split disimpan: {out} (folds={len(split)})")
```

4.1.3 *Pre-Processing* dan Augmentasi Data

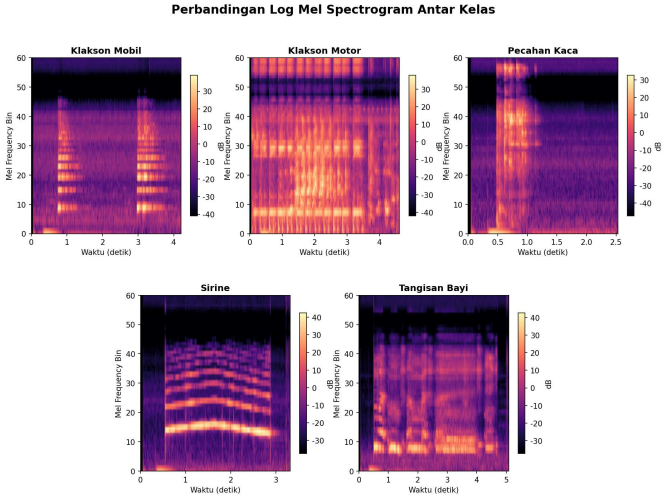
4.1.3.1 *Pre-Processing* Data

Tahap *pre-processing* bertujuan untuk menyiapkan data audio mentah agar sesuai dengan kebutuhan model pembelajaran mesin. Proses ini meliputi normalisasi amplitudo, penghilangan kebisingan latar belakang, dan

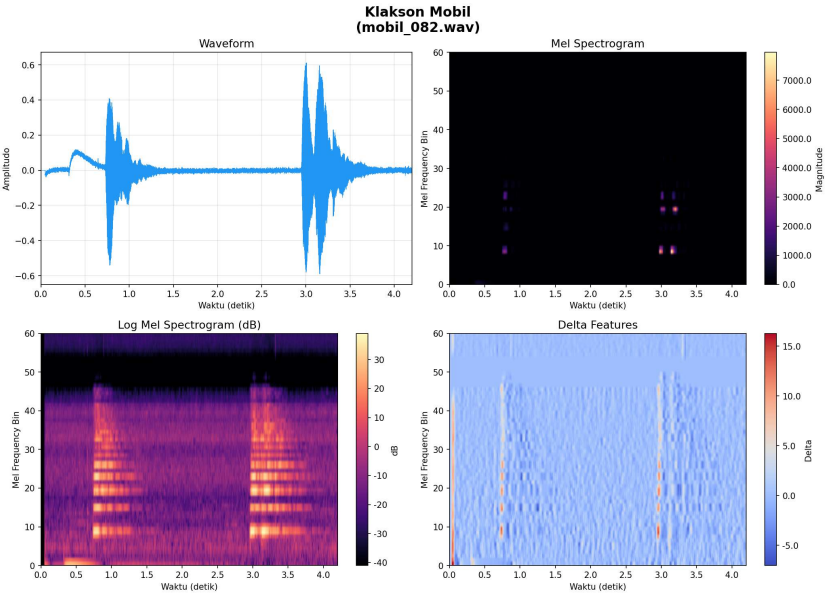
pemotongan atau pengisian ulang (*padding*) durasi audio agar seragam. Setiap sampel audio diubah menjadi representasi spektrogram. Berikut adalah contoh spektrogram dari beberapa sampel audio yang telah diproses, seperti yang ditunjukkan pada Gambar 4.1 hingga Gambar 4.7.



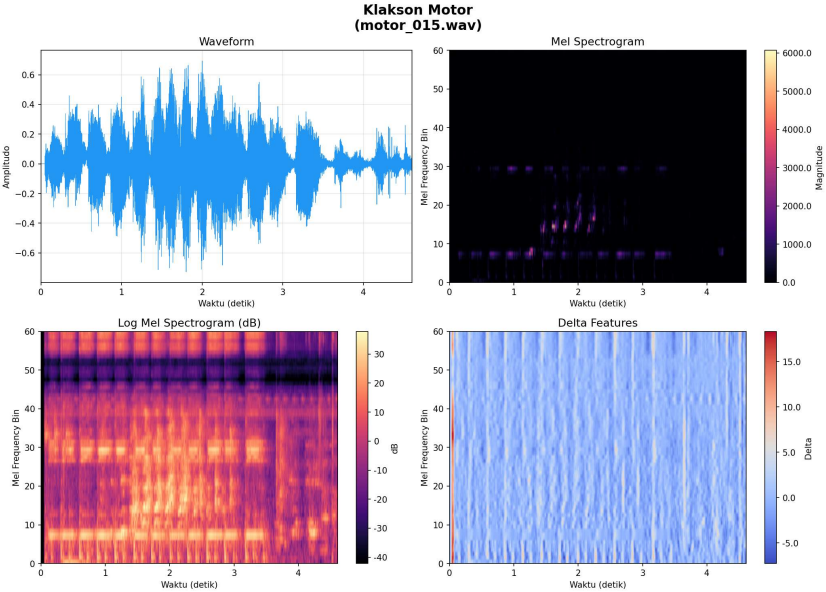
Gambar 4.1 Alur Pipeline Pre-processing Data Audio



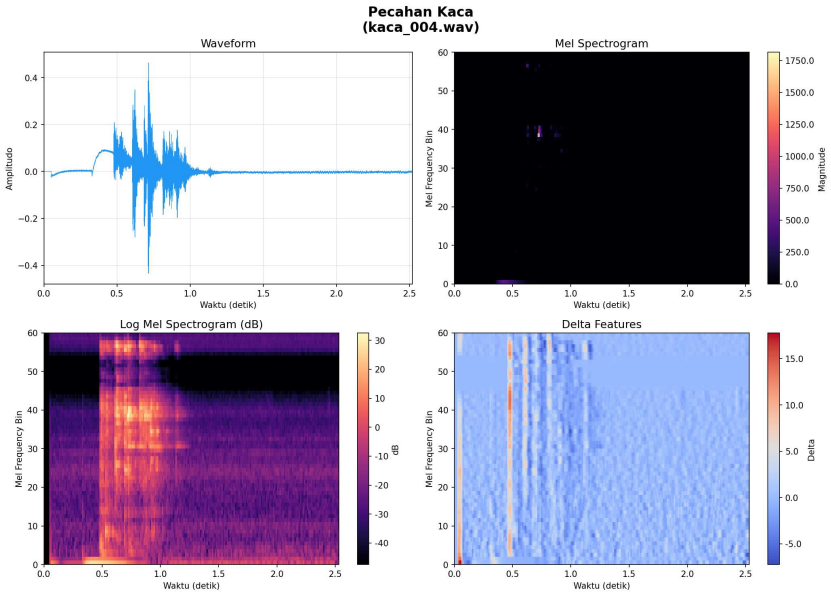
Gambar 4.2 Perbandingan Log-Mel Spectrogram



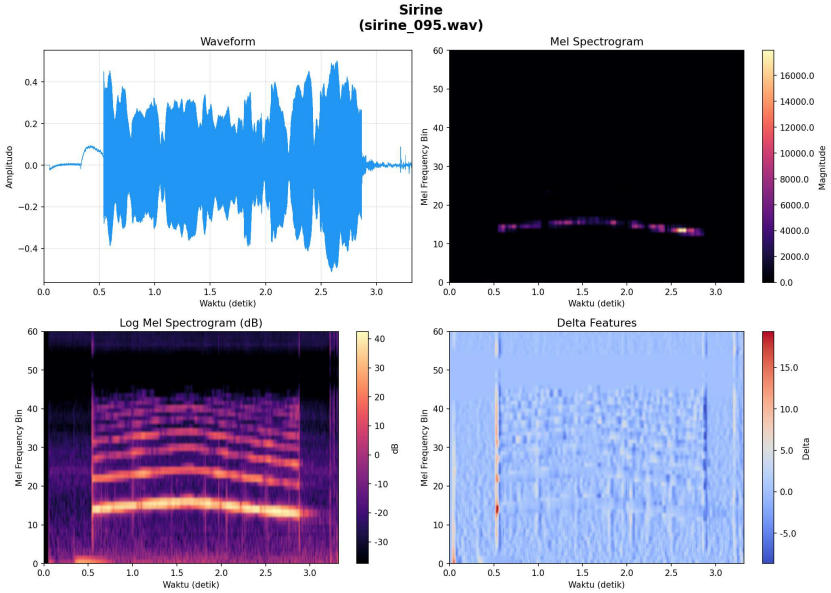
Gambar 4.3 Klakson Mobil



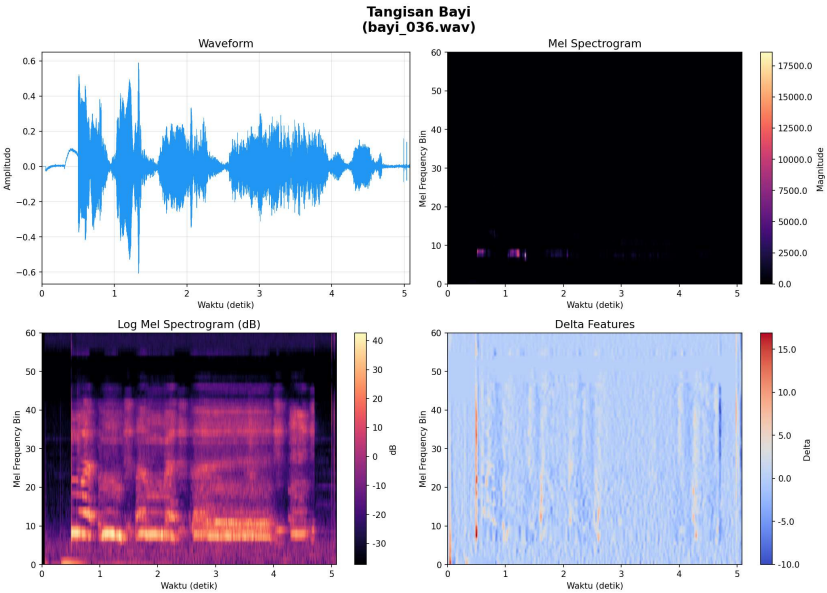
Gambar 4.4 Klakson Motor



Gambar 4.5 Pecahan Kaca



Gambar 4.6 Sirine



Gambar 4.7 Tangisan Bayi

Spektrogram ini kemudian dinormalisasi untuk memastikan konsistensi skala antar sampel. Fitur yang digunakan dalam penelitian ini adalah *log-Mel spectrogram*, yang diekstraksi menggunakan jendela Hamming dengan ukuran 1024 sampel dan tumpang tindih 50%. Jumlah *Mel filter* yang digunakan adalah 60, menghasilkan matriks fitur berukuran 60 x T, di mana T adalah jumlah bingkai waktu yang bervariasi tergantung pada durasi sampel audio.

Kode 4.2 Implementasi kode pre-processing data audio

```
import torch
import torchaudio
import random
import torch.nn.functional as F
from torch.utils.data import Dataset


class AudioDataset(Dataset):
    def __init__(self, root_dir, fold, split_type='train',
     segment_length=41):
        self.segment_length = segment_length
        self.mel_transform = torchaudio.transforms.MelSpectrogram(
            sample_rate=44100,
            n_fft=1024,
            hop_length=512,
            n_mels=60
        )
```

```
        self.amp2db = torchaudio.transforms.AmplitudeToDB(
            stype="power",
            top_db=80
        )
        self.delta = torchaudio.transforms.ComputeDeltas()

    def __getitem__(self, idx):
        path, label_id = self.samples[idx]
        wav, sr = torchaudio.load(path)
        mel = self.mel_transform(wav)
        logmel = self.amp2db(mel)

        T = logmel.shape[2]
        if T < self.segment_length:
            logmel = F.pad(logmel, (0, self.segment_length - T))
            T = self.segment_length

        start = 0 if T == self.segment_length else random.randint(
            0, T - self.segment_length
        )
        seg = logmel[:, :, start:start + self.segment_length]
        deltas = self.delta(seg)
        stacked = torch.cat((seg, deltas), dim=0)

        return stacked, label_id, path
```

4.1.3.2 Augmentasi Data

Untuk meningkatkan keragaman data latih dan mengurangi *overfitting*, diterapkan augmentasi data. Manipulasi pada domain waktu (misalnya pergeseran waktu) dilakukan pada gelombang suara sebelum ekstraksi fitur, sedangkan augmentasi berbasis spektrogram (misalnya *SpecAugment*) diterapkan setelah fitur *log-Mel* diperoleh. Pembagian data latih/validasi mengacu pada berkas konfigurasi `split.json`; augmentasi hanya diterapkan pada himpunan data latih.

Teknik yang digunakan meliputi *time-shifting*, penambahan derau latar (*adding background noise*) dengan SNR acak, serta *spectrogram augmentation* (masking frekuensi dan waktu). Selama pelatihan, setiap teknik diaktifkan secara acak dengan probabilitas 50% per sampel. Implementasi ringkas augmentasi dapat dilihat pada Kode 4.3.

Kode 4.3 Implementasi Augmentasi Data

```
import random

import numpy as np
def time_shift_tensor(waveform: torch.Tensor, max_shift_ratio: float
     = 0.2) -> torch.Tensor:
    if max_shift_ratio <= 0 or waveform.size(1) < 2:
        return waveform
```

```
    shift = int(waveform.size(1) * max_shift_ratio * (random.random() * 2 - 1))
    if shift == 0:
        return waveform
    if shift > 0:
        pad = torch.zeros(waveform.size(0), shift, dtype=waveform.dtype)
        waveform = torch.cat([pad, waveform[:, :-shift]], dim=1)
    else:
        shift = -shift
        pad = torch.zeros(waveform.size(0), shift, dtype=waveform.dtype)
        waveform = torch.cat([waveform[:, shift:], pad], dim=1)
    return waveform

def add_background_noise_tensor(waveform: torch.Tensor, snr_db_range=(15, 25)) -> torch.Tensor:
    num_channels, num_samples = waveform.shape
    if num_samples == 0:
        return waveform
    snr_db = np.random.uniform(snr_db_range[0], snr_db_range[1])
    signal_power = waveform.pow(2).mean().item()
    if signal_power <= 1e-12:
        return waveform
    noise_power = signal_power / (10 ** (snr_db / 10))
    # Generate Gaussian noise per channel
    noise = torch.randn_like(waveform)
    current_noise_power = noise.pow(2).mean().item()
    if current_noise_power <= 1e-12:
        return waveform
    scale = (noise_power / current_noise_power) ** 0.5
    waveform_noisy = waveform + noise * scale
    return waveform_noisy

def spec_augment_tensor(mel: torch.Tensor,
                        freq_mask_param: int = 8,
                        time_mask_param: int = 10,
                        num_freq_masks: int = 1,
                        num_time_masks: int = 1) -> torch.Tensor:
    channels, freq, time = mel.shape
    # Frequency masks
    for _ in range(num_freq_masks):
        f = random.randint(0, freq_mask_param)
        if f > 0 and f < freq:
            f0 = random.randint(0, freq - f)
            mel[:, f0:f0+f, :] = 0
    # Time masks
    for _ in range(num_time_masks):
        t = random.randint(0, time_mask_param)
        if t > 0 and t < time:
            t0 = random.randint(0, time - t)
            mel[:, :, t0:t0+t] = 0
    return mel
```

4.1.4 Pelatihan Model

Arsitektur model yang digunakan dalam penelitian ini adalah Piczak-CNN, yaitu sebuah model jaringan saraf konvolusi yang dirancang secara khusus untuk keperluan klasifikasi suara lingkungan. Model ini menerima masukan berupa representasi spektrogram *log-Mel* yang telah diproses pada tahap sebelumnya.

Representasi spektrogram *log-Mel* yang dihasilkan memiliki dimensi 2 ×

60 × 41. Dimensi pertama (nilai 2) merepresentasikan dua saluran fitur yaitu *log-Mel spectrogram* statis dan turunannya (*delta*). Kedua saluran digabungkan melalui operasi concatenation. Saluran pertama menangkap energi spektral, sedangkan saluran kedua menangkap laju perubahan fitur spektral terhadap waktu, yang membantu membedakan pola transien seperti onset dan decay pada kelas-kelas suara. Pendekatan dua saluran ini analogi dengan representasi citra RGB dalam visi komputer. Dimensi kedua (nilai 60) menyatakan jumlah *Mel filter bank*, dan dimensi ketiga (nilai 41) menyatakan jumlah bingkai waktu dari segmentasi sinyal audio.

Dimensi pertama dengan nilai 2 merepresentasikan dua saluran fitur berbeda yang diekstraksi dari sinyal audio. Saluran pertama berisi *log-Mel spectrogram* yang merepresentasikan distribusi energi spektral statis pada domain frekuensi versus waktu. Saluran kedua berisi fitur turunan pertama (*delta*) yang merepresentasikan laju perubahan karakteristik spektral sepanjang dimensi waktu. Pendekatan dua saluran ini analogi dengan representasi citra berwarna dalam visi komputer yang menggunakan tiga saluran (merah, hijau, biru). Fitur *delta* memberikan informasi transisional yang penting untuk membedakan dinamika temporal suara, khususnya pada fenomena seperti serangan (*onset*) dan peluruhan amplitudo, yang membedakan kelas-kelas suara seperti klakson, sirene, pecahan kaca, dan tangisan bayi. Kedua saluran fitur ini digabungkan melalui operasi penggabungan (*concatenation*) pada dimensi saluran, seperti yang ditunjukkan pada Kode 4.2 baris `stacked = torch.cat((seg, deltas), dim=0)`.

Dimensi kedua bernilai 60, yang menyatakan jumlah *filter bank* pada skala Mel yang digunakan dalam ekstraksi fitur spektrogram. Dimensi ketiga bernilai 41, yang menunjukkan jumlah bingkai waktu (*frame*) yang dihasilkan dari segmentasi sinyal audio menggunakan jendela geser (*sliding window*). Jumlah bingkai waktu dapat bervariasi tergantung durasi sampel audio, namun dalam penelitian ini dibatasi pada 41 bingkai untuk memastikan konsistensi dimensi

masukan model akibat adanya operasi *padding* pada sampel lebih pendek.

Arsitektur Piczak-CNN yang diimplementasikan terdiri atas dua blok konvolusi dan dua lapisan *fully connected*. Blok konvolusi pertama memiliki 80 filter dengan ukuran kernel 57 × 6, sedangkan blok konvolusi kedua memiliki 80 filter dengan ukuran kernel 1 × 3. Masing-masing blok konvolusi diikuti oleh fungsi aktivasi ReLU dan operasi *max-pooling* yang berfungsi untuk mengekstraksi fitur serta mereduksi dimensi representasi. Lapisan *dropout* dengan probabilitas 0,5 diterapkan untuk mencegah *overfitting*. Lapisan *fully connected* pertama memiliki 5000 unit neuron dengan aktivasi ReLU, diikuti oleh lapisan *fully connected* kedua dengan jumlah unit yang sama. Lapisan keluaran memiliki jumlah unit sebanyak jumlah kelas target, yaitu 5 kelas. Implementasi arsitektur model secara lengkap ditunjukkan pada Kode 4.4.

Kode 4.4 Arsitektur Model Piczak-CNN

```
import torch
import torch.nn as nn
from torchinfo import summary

class PiczakCNN(nn.Module):
        def __init__(self, num_classes, input_shape=(60, 41),
     dropout_rate=0.5):
                super(PiczakCNN, self).__init__()

                self.conv1 = nn.Sequential(
                        nn.Conv2d(in_channels=2, out_channels=80,
     kernel_size=(57, 6)),
                        nn.ReLU()
                )
                self.pool1 = nn.MaxPool2d(
                        kernel_size=(4, 3), stride=(1, 3)
                )
                self.dropout1 = nn.Dropout(dropout_rate)

                self.conv2 = nn.Sequential(
                        nn.Conv2d(in_channels=80, out_channels=80,
     kernel_size=(1, 3)),
                        nn.ReLU()
                )
                self.pool2 = nn.MaxPool2d(
                        kernel_size=(1, 3), stride=(1, 3)
                )

                self.flatten = nn.Flatten()

                with torch.no_grad():
                        dummy_input = torch.zeros(1, 2, *input_shape)
                        x = self.conv1(dummy_input)
                        x = self.pool1(x)
                        x = self.conv2(x)
```

```
36                          x = self.pool2(x)
37                          x = self.flatten(x)
38                          fc_input_features = x.shape[1]
39
40                  self.fc1_block = nn.Sequential(
41                          nn.Dropout(dropout_rate),
42                          nn.Linear(fc_input_features, 5000),
43                          nn.ReLU(),
44                          nn.Dropout(dropout_rate)
45                  )
46
47                  self.fc2_block = nn.Sequential(
48                          nn.Linear(5000, 5000),
49                          nn.ReLU()
50                  )
51
52                  self.output_layer = nn.Linear(5000, num_classes)
53
54          def forward(self, x):
55                  x = self.conv1(x)
56                  x = self.pool1(x)
57                  x = self.dropout1(x)
58
59                  x = self.conv2(x)
60                  x = self.pool2(x)
61
62                  x = self.flatten(x)
63
64                  x = self.fc1_block(x)
65                  x = self.fc2_block(x)
66
67                  x = self.output_layer(x)
68                  return x
69
70  if __name__ == '__main__':
71          NUM_CLASSES = 10
72          INPUT_SHAPE = (60, 41)
73
74          model = PiczakCNN(
75                  num_classes=NUM_CLASSES,
76                  input_shape=INPUT_SHAPE
77          )
```

Model dilatih menggunakan konfigurasi hiperparameter baseline yang telah ditentukan berdasarkan eksperimen awal, yaitu menggunakan optimizer SGD dengan laju pembelajaran 0,002, momentum 0,9, dan Nesterov diaktifkan. Batch size yang digunakan adalah 32, dengan jumlah epoch maksimum sebanyak 50. Selain itu, digunakan scheduler ReduceLROnPlateau untuk menyesuaikan laju pembelajaran secara dinamis berdasarkan kinerja validasi, dengan faktor pengurangan sebesar 0,5 dan kesabaran selama 3 epoch tanpa perbaikan. Early stopping juga diterapkan dengan kesabaran selama 5 epoch tanpa perbaikan pada metrik validasi untuk mencegah overfitting. Seluruh konfigurasi hiperparameter pelatihan disajikan pada Tabel 4.1.

Tabel 4.1 Konfigurasi Hyperparameter Pelatihan Model

Hyperparameter	**Nilai**
Optimizer	SGD
Learning Rate	0,002
Momentum	0,9
Nesterov	True
Batch Size	32
Maximum Epochs	50
Scheduler	ReduceLROnPlateau
Scheduler Factor	0,5
Scheduler Patience	3 epochs
Minimum Learning Rate	1e-4
Early Stopping Patience	5 epochs

Kode 4.5 Implementasi Pelatihan Model

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torch.optim.lr_scheduler import ReduceLROnPlateau

model = PiczakCNN(
        num_classes=num_classes,
        input_shape=(60, 41)
).to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(
        model.parameters(), lr=0.002,
        momentum=0.9, nesterov=True
)

scheduler = ReduceLROnPlateau(
        optimizer, mode="min", factor=0.5,
        patience=3, min_lr=1e-4
)

early_patience = 5
early_min_delta = 1e-4
best_val_loss = float('inf')
epochs_no_improve = 0

for epoch in range(epochs):
        model.train()
        running_loss = 0.0
        for specs, labels, _ in train_loader:
                specs = specs.to(device)
                labels = labels.to(device).long()
                optimizer.zero_grad()
                outputs = model(specs)
                loss = criterion(outputs, labels)
                loss.backward()
                optimizer.step()
                running_loss += loss.item() * specs.size(0)
        train_loss = running_loss / len(train_dataset)

        model.eval()
        val_running_loss = 0.0
```

```
        with torch.no_grad():
                for specs, labels, _ in val_loader:
                        specs = specs.to(device)
                        labels = labels.to(device).long()
                        outputs = model(specs)
                        loss = criterion(outputs, labels)
                        val_running_loss += loss.item() * specs.size(0)
        val_loss = val_running_loss / len(val_dataset)

        scheduler.step(val_loss)

        if val_loss < best_val_loss - early_min_delta:
                best_val_loss = val_loss
                epochs_no_improve = 0
        else:
                epochs_no_improve += 1
                if epochs_no_improve >= early_patience:
                        print("Early stopping triggered.")
                        break
```

Evaluasi model dilakukan menggunakan skema validasi silang (*cross-validation*) dengan pembagian *fold* yang konsisten untuk seluruh model. Metrik utama yang digunakan adalah *accuracy*, karena dataset yang digunakan memiliki distribusi kelas yang seimbang. Selain itu, metrik tambahan seperti *F1-score*, presisi, dan *recall* juga dihitung untuk memberikan gambaran yang lebih komprehensif mengenai kinerja model, terutama dalam konteks klasifikasi multi-kelas. Proses evaluasi dilakukan dengan langkah-langkah sebagai berikut:

1. Setiap *fold* digunakan sebagai data validasi secara bergantian, sedangkan *fold* lainnya digunakan sebagai data pelatihan.
2. Setelah proses pelatihan pada setiap *fold* selesai, model diuji menggunakan data validasi untuk menghitung seluruh metrik evaluasi.
3. Hasil evaluasi dari seluruh *fold* kemudian dirata-ratakan untuk memperoleh gambaran umum mengenai kinerja model secara keseluruhan.

Selain itu, seluruh hasil pelatihan dan evaluasi dicatat menggunakan *Weights & Biases* (WandB) untuk memantau proses pelatihan secara *real-time* serta memfasilitasi analisis lebih lanjut setelah pelatihan selesai.

4.1.4.1 Pengujian Model

Proses pemilihan *hyperparameter* dilaksanakan secara sistematis dengan memanfaatkan skema *5-fold cross-validation*. Pada setiap kombinasi *hyperparameter*, model dilatih pada seluruh lima *fold* dan nilai akurasi validasi tertinggi dari setiap *fold* dicatat. Nilai rata-rata akurasi dari kelima *fold* tersebut kemudian dijadikan sebagai metrik pembanding antar kombinasi. Adapun *hyperparameter* yang diujikan mencakup jenis *optimizer* (SGD dengan laju pembelajaran 0,002 dan Adam dengan laju pembelajaran 0,001), nilai *dropout rate* (0; 0,1; 0,2; dan 0,3), serta penggunaan augmentasi data (dengan dan tanpa augmentasi). Dari variasi tersebut, diperoleh total 16 kombinasi *hyperparameter* yang diujikan. Hasil eksperimen tahap awal disajikan pada Tabel 4.2.

Tabel 4.2 Hasil Eksperimen Pemilihan *Hyperparameter*

No	***Opt.***	***LR***	***Drop.***	**Aug.**	**Akurasi**
0	SGD	0,002	0,5	Ya	0,598 (baseline)
1	SGD	0,002	0,0	Ya	0,728
2	SGD	0,002	0,0	Tidak	**0,822**
3	SGD	0,002	0,1	Ya	0,772
4	SGD	0,002	0,1	Tidak	**0,822**
5	SGD	0,002	0,2	Ya	0,740
6	SGD	0,002	0,2	Tidak	0,812
7	SGD	0,002	0,3	Ya	0,726
8	SGD	0,002	0,3	Tidak	**0,818**
9	Adam	0,001	0,0	Ya	0,712
10	Adam	0,001	0,0	Tidak	0,722
11	Adam	0,001	0,1	Ya	0,702
12	Adam	0,001	0,1	Tidak	0,770

Tabel 4.2 Hasil Eksperimen Pemilihan *Hyperparameter* (lanjutan)

No	***Opt.***	***LR***	***Drop.***	**Aug.**	**Akurasi**
13	Adam	0,001	0,2	Ya	0,654
14	Adam	0,001	0,2	Tidak	0,740
15	Adam	0,001	0,3	Ya	0,680
16	Adam	0,001	0,3	Tidak	0,578

Berdasarkan hasil eksperimen yang disajikan pada Tabel 4.2, dapat diamati bahwa penggunaan *optimizer* SGD secara konsisten menghasilkan performa yang lebih baik dibandingkan Adam pada seluruh variasi *dropout rate* maupun konfigurasi augmentasi yang diujikan. Di samping itu, pelatihan tanpa penerapan augmentasi data menunjukkan pencapaian akurasi yang lebih tinggi dibandingkan pelatihan yang disertai augmentasi. Berdasarkan temuan tersebut, tiga kombinasi *hyperparameter* dengan akurasi tertinggi dipilih untuk diverifikasi lebih lanjut, yaitu:

- SGD, *learning rate* 0,002, *dropout rate* 0,0, tanpa augmentasi (akurasi 0,822);
- SGD, *learning rate* 0,002, *dropout rate* 0,1, tanpa augmentasi (akurasi 0,822);
- SGD, *learning rate* 0,002, *dropout rate* 0,3, tanpa augmentasi (akurasi 0,818).

Guna memvalidasi konsistensi hasil yang diperoleh, ketiga kombinasi terbaik tersebut dilatih ulang sebanyak dua kali tambahan, sehingga secara keseluruhan terdapat tiga percobaan untuk setiap kombinasi. Nilai rata-rata akurasi dari ketiga percobaan tersebut selanjutnya digunakan sebagai metrik akhir dalam menentukan konfigurasi *hyperparameter* yang paling optimal. Hasil verifikasi secara lengkap disajikan pada Tabel 4.3.

Tabel 4.3 Hasil Verifikasi 3 Kombinasi *Hyperparameter* Terbaik

Dropout Rate	**Percobaan 1**	**Percobaan 2**	**Percobaan 3**	**Rata-rata**
0,0	0,822	0,812	0,836	**0,823**
0,1	0,822	0,808	0,724	0,785
0,3	0,818	0,804	0,810	0,811

Berdasarkan hasil verifikasi yang disajikan pada Tabel 4.3, dapat disimpulkan bahwa kombinasi *hyperparameter* yang menghasilkan performa paling optimal adalah *optimizer* SGD dengan laju pembelajaran sebesar 0,002, nilai *dropout rate* 0,0, serta tanpa penerapan augmentasi data. Kombinasi tersebut mencapai nilai akurasi rata-rata tertinggi, yaitu 0,823, yang diperoleh dari tiga kali percobaan. Meskipun kombinasi dengan *dropout rate* 0,3 memperoleh hasil yang hampir mendekati, yakni sebesar 0,811, kombinasi dengan *dropout rate* 0,0 tetap dipilih dengan pertimbangan bahwa konfigurasi tersebut menunjukkan konsistensi performa yang lebih baik pada seluruh percobaan yang dilakukan. Oleh karena itu, konfigurasi *hyperparameter* ini ditetapkan sebagai konfigurasi utama yang digunakan untuk proses evaluasi model pada tahap selanjutnya.

4.1.4.2 Perbandingan Model

Setelah konfigurasi *hyperparameter* optimal ditetapkan, tahap selanjutnya adalah melakukan perbandingan performa model Piczak-CNN dengan arsitektur lain yang umum digunakan dalam domain klasifikasi suara lingkungan. Model pembanding yang dipilih adalah PANNs (*Pretrained Audio Neural Networks*) CNN14, yaitu arsitektur jaringan saraf konvolusi yang telah dilatih sebelumnya pada dataset AudioSet berskala besar. Pemilihan model tersebut dilandasi oleh kapabilitasnya dalam melakukan ekstraksi fitur audio secara hierarkis serta performanya yang telah divalidasi pada berbagai tugas klasifikasi suara di penelitian-penelitian terdahulu.

Evaluasi perbandingan dilakukan dengan menguji model PANNs CNN14 pada dataset yang sama dengan yang digunakan untuk melatih model Piczak-CNN. Kedua model dievaluasi menggunakan metrik yang seragam, yaitu akurasi, *F1-score*, presisi, dan *recall*. Hasil evaluasi dari kedua arsitektur tersebut kemudian dianalisis secara komparatif untuk mengidentifikasi kelebihan dan kekurangan masing-masing model dalam konteks klasifikasi suara lingkungan.

Arsitektur PANNs CNN14 yang digunakan dalam penelitian ini merupakan model *deep convolutional neural network* yang terdiri atas enam blok konvolusi dengan jumlah filter yang meningkat secara bertahap, yaitu 64, 128, 256, 512, 1024, dan 2048. Setiap blok konvolusi terdiri dari dua lapisan konvolusi 2D dengan ukuran kernel 3×3, masing-masing diikuti oleh *batch normalization* dan fungsi aktivasi ReLU, serta operasi *average pooling* untuk mereduksi dimensi spasial. Lapisan *dropout* dengan probabilitas 0,2 diterapkan setelah setiap blok konvolusi. Setelah melewati seluruh blok konvolusi, dilakukan *global pooling* yang menggabungkan hasil *mean pooling* dan *max pooling* pada dimensi waktu. Keluaran dari *global pooling* kemudian diteruskan ke lapisan *fully connected* dengan 2048 unit neuron untuk menghasilkan *embedding* audio.

Kode 4.6 Arsitektur Model PANNs CNN14

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class ConvBlock(nn.Module):
        def __init__(self, in_channels, out_channels):
                super().__init__()
                self.conv1 = nn.Conv2d(
                        in_channels, out_channels,
                        kernel_size=3, padding=1, bias=False
                )
                self.bn1 = nn.BatchNorm2d(out_channels)
                self.conv2 = nn.Conv2d(
                        out_channels, out_channels,
                        kernel_size=3, padding=1, bias=False
                )
                self.bn2 = nn.BatchNorm2d(out_channels)

        def forward(self, x, pool_size=(2, 2)):
                x = F.relu(self.bn1(self.conv1(x)))
```

```
21                 x = F.relu(self.bn2(self.conv2(x)))
22                 x = F.avg_pool2d(x, kernel_size=pool_size)
23                 return x
24
25
26 class Cnn14(nn.Module):
27         def __init__(self, sample_rate=44100,
28                                 mel_bins=64, classes_num=527):
29                 super().__init__()
30                 self.bn0 = nn.BatchNorm2d(64)
31                 self.conv_block1 = ConvBlock(1, 64)
32                 self.conv_block2 = ConvBlock(64, 128)
33                 self.conv_block3 = ConvBlock(128, 256)
34                 self.conv_block4 = ConvBlock(256, 512)
35                 self.conv_block5 = ConvBlock(512, 1024)
36                 self.conv_block6 = ConvBlock(1024, 2048)
37                 self.fc1 = nn.Linear(2048, 2048, bias=True)
38                 self.fc_audioset = nn.Linear(
39                         2048, classes_num, bias=True
40                 )
41
42         def forward(self, x):
43                 if x.shape[1] == 2:
44                         x = x.mean(dim=1, keepdim=True)
45                 x = x.transpose(2, 3)
46                 x = x.transpose(1, 3)
47                 x = self.bn0(x)
48                 x = x.transpose(1, 3)
49
50                 x = self.conv_block1(x, pool_size=(2, 2))
51                 x = F.dropout(x, p=0.2, training=self.training)
52                 x = self.conv_block2(x, pool_size=(2, 2))
53                 x = F.dropout(x, p=0.2, training=self.training)
54                 x = self.conv_block3(x, pool_size=(2, 2))
55                 x = F.dropout(x, p=0.2, training=self.training)
56                 x = self.conv_block4(x, pool_size=(2, 2))
57                 x = F.dropout(x, p=0.2, training=self.training)
58                 x = self.conv_block5(x, pool_size=(2, 2))
59                 x = F.dropout(x, p=0.2, training=self.training)
60                 x = self.conv_block6(x, pool_size=(1, 1))
61                 x = F.dropout(x, p=0.2, training=self.training)
62
63                 x = torch.mean(x, dim=3)
64                 (x1, _) = torch.max(x, dim=2)
65                 x2 = torch.mean(x, dim=2)
66                 x = x1 + x2
67
68                 x = F.dropout(x, p=0.5, training=self.training)
69                 x = F.relu(self.fc1(x))
70                 embedding = F.dropout(
71                         x, p=0.5, training=self.training
72                 )
73                 output = self.fc_audioset(embedding)
74                 return output
75
76
77 class PANNsCNN14FineTune(nn.Module):
78         def __init__(self, num_classes=5,
79                                 pretrained_path=None,
80                                 freeze_base=True):
81                 super().__init__()
82                 self.base_model = Cnn14(classes_num=527)
83
84                 if pretrained_path is not None:
85                         checkpoint = torch.load(
86                                 pretrained_path, map_location='cpu',
87                                 weights_only=False
88                         )
89                         self.base_model.load_state_dict(
```

```
                                checkpoint['model'], strict=False
                        )

                self.base_model.bn0 = nn.BatchNorm2d(60)

                if freeze_base:
                        for param in self.base_model.parameters():
                                param.requires_grad = False
                        for param in
    self.base_model.bn0.parameters():
                                param.requires_grad = True

                self.base_model.fc_audioset = nn.Identity()

                self.classifier = nn.Sequential(
                        nn.Linear(2048, 512),
                        nn.ReLU(),
                        nn.Dropout(0.3),
                        nn.Linear(512, num_classes)
                )

        def forward(self, x):
                embedding = self.base_model(x)
                output = self.classifier(embedding)
                return output
```

Untuk memberikan gambaran yang jelas mengenai perbedaan arsitektur kedua model, Tabel 4.4 menyajikan perbandingan spesifikasi arsitektur antara Piczak-CNN dengan konfigurasi *hyperparameter* terbaik yang telah ditetapkan sebelumnya dan PANNs CNN14. Tabel tersebut mencakup aspek-aspek seperti jumlah blok konvolusi, jumlah filter, ukuran kernel, jenis *pooling*, penggunaan *batch normalization*, nilai *dropout rate*, jumlah unit pada lapisan *fully connected*, status *pre-trained*, jumlah saluran masukan, jenis *optimizer*, nilai *learning rate*, serta jumlah kelas keluaran.

Tabel 4.4 Perbandingan Arsitektur Model Piczak-CNN dan PANNs CNN14

Aspek	**Piczak-CNN**	**PANNs CNN14**
Jumlah blok konvolusi	2	6
Jumlah filter	80	64–2048
Ukuran kernel	57×6, 1×3	3×3
Pooling	*Max pooling*	*Average pooling*
Batch normalization	Tidak	Ya
Dropout rate	0,5	0,2 (konvolusi), 0,5 (FC)

Tabel 4.4 Perbandingan Arsitektur Model Piczak-CNN dan PANNs CNN14 (lanjutan)

Aspek	**Piczak-CNN**	**PANNs CNN14**
Jumlah unit FC	5000	2048
Pre-trained	Tidak	Ya (AudioSet)
Jumlah saluran masukan	2 (log-Mel + delta)	1 (log-Mel)
Optimizer	SGD	SGD
Learning rate	0,002	0,002
Jumlah kelas keluaran	5	5

Selanjutnya, hasil evaluasi perbandingan performa kedua model yaitu Piczak-CNN dengan konfigurasi *hyperparameter* terbaik yang telah ditetapkan sebelumnya (SGD, *learning rate* 0,002, *dropout rate* 0,0, tanpa augmentasi) dan model PANNs CNN14 yang di-*fine-tune* pada dataset yang sama disajikan pada Tabel 4.5. Tabel tersebut menyajikan nilai akurasi, *F1-score*, dan presisi untuk masing-masing model pada setiap *fold* serta nilai rata-rata keseluruhan.

Tabel 4.5 Perbandingan Performa Model Piczak-CNN dan PANNs CNN14

Fold	**Piczak-CNN**			**PANNs CNN14**		
	Akurasi	***F1***	**Presisi**	**Akurasi**	***F1***	**Presisi**
1	0,87	0,872	0,8757	0,57	0,5801	0,6069
2	0,76	0,7186	0,8149	0,65	0,5979	0,6704
3	**0,90**	0,8935	0,9002	0,71	0,6926	0,7263
4	0,86	0,8569	0,8615	0,64	0,6336	0,6838
5	0,79	0,7556	0,7942	0,63	0,6119	0,6641
Rata-rata	**0,836**	**0,81932**	**0,8493**	**0,64**	**0,62322**	**0,6703**

Berdasarkan Tabel 4.5, perbandingan performa antara model Piczak-CNN dengan konfigurasi *hyperparameter* terbaik (SGD, *learning rate* 0,002,

dropout rate 0,0, tanpa augmentasi) dan model PANNs CNN14 yang di-*fine-tune* pada dataset yang sama menunjukkan bahwa kedua model memiliki karakteristik performa yang berbeda. Analisis lebih lanjut mengenai kelebihan dan kekurangan masing-masing model dibahas pada subbab pembahasan.

4.1.5 Evaluasi Studi Ablasi

Studi ablasi dilaksanakan untuk mengevaluasi kontribusi setiap komponen arsitektur Piczak-CNN terhadap performa model secara keseluruhan. Komponen yang dievaluasi mencakup mekanisme *dropout*, teknik augmentasi data, dan pemilihan algoritma optimisasi. Analisis dilakukan secara bertahap mengikuti skenario pada Bab III, dimulai dari konfigurasi *baseline*, kemudian variasi *dropout*, dilanjutkan evaluasi augmentasi, dan terakhir perbandingan *optimizer*.

4.1.5.1 Baseline Model

Studi ablasi dimulai dengan penetapan model dasar (*baseline*) yang berfungsi sebagai acuan dalam evaluasi kontribusi komponen-komponen berbeda. Model dasar dikonfigurasi dengan parameter berikut: *optimizer* SGD, laju pembelajaran 0,002, *dropout rate* 0,5, dan penerapan augmentasi data. Performa model dasar ini kemudian dijadikan referensi utama untuk membandingkan tingkat peningkatan atau penurunan akurasi yang dihasilkan dari penghapusan setiap komponen pada tahap selanjutnya.

Berdasarkan hasil eksperimen awal pada Tabel 4.2, model dasar dengan konfigurasi tersebut mencapai nilai akurasi rata-rata sebesar 0,598. Nilai akurasi ini menjadi titik tolak fundamental dalam studi ablasi untuk mengukur kontribusi faktual dari masing-masing komponen terhadap peningkatan performa model secara menyeluruh.

4.1.5.2 Eksperimentasi Dropout

Tahap analisis pertama mengikuti skenario 2 pada Bab III, yaitu memvariasikan *dropout rate* (0,0; 0,1; 0,2; 0,3) dengan konfigurasi lain tetap seperti *baseline* (SGD, laju pembelajaran 0,002, augmentasi aktif). Berdasarkan Tabel 4.2, nilai terbaik pada skenario ini diperoleh saat *dropout rate* 0,0 dengan akurasi tertinggi dibandingkan variasi lainnya. Hasil ini menunjukkan bahwa penghapusan *dropout* dari 0,5 ke 0,0 meningkatkan kemampuan model dalam mempelajari pola data latih pada dataset penelitian ini.

4.1.5.3 Eksperimentasi Augmentasi Data

Tahap analisis kedua mengikuti skenario 3 pada Bab III, yaitu membandingkan augmentasi aktif dan nonaktif menggunakan nilai *dropout* terbaik dari skenario 2 (*dropout rate* 0,0), dengan *optimizer* SGD dan laju pembelajaran 0,002. Hasil pada Tabel 4.2 menunjukkan bahwa saat augmentasi dinonaktifkan, akurasi tetap menjadi yang terbaik. Temuan ini mengindikasikan bahwa pada dataset penelitian ini, augmentasi yang diterapkan belum memberikan keuntungan performa dan cenderung menurunkan akurasi.

4.1.5.4 Eksperimentasi Optimizer

Tahap analisis ketiga mengikuti skenario 4 pada Bab III, yaitu membandingkan *optimizer* dengan konfigurasi terbaik dari skenario 3. Pada kondisi *dropout rate* 0,0 dan tanpa augmentasi, penggunaan SGD menghasilkan akurasi terbaik, sedangkan Adam menghasilkan akurasi yang lebih rendah (Tabel 4.2). Dengan demikian, SGD terbukti lebih sesuai untuk konfigurasi dan karakteristik data pada penelitian ini.

4.2 Evaluasi

Tahap evaluasi dilaksanakan untuk membandingkan performa model Piczak-CNN dengan konfigurasi *hyperparameter* optimal yang telah ditetapkan

(SGD, laju pembelajaran 0,002, *dropout rate* 0,0, tanpa augmentasi) terhadap model PANNs CNN14 yang di-*fine-tune* pada dataset yang sama. Perbandingan dilakukan menggunakan metrik akurasi, *F1-score*, dan presisi untuk setiap *fold* serta nilai rata-rata keseluruhan. Hasil evaluasi menunjukkan bahwa model Piczak-CNN mencapai performa yang lebih unggul dibandingkan model PANNs CNN14 pada seluruh metrik yang digunakan.

Model Piczak-CNN mencapai nilai akurasi rata-rata 0,836, *F1-score* rata-rata 0,81932, dan presisi rata-rata 0,8493. Sebaliknya, model PANNs CNN14 yang di-*fine-tune* pada dataset yang sama memperoleh nilai akurasi rata-rata 0,64, *F1-score* rata-rata 0,62322, dan presisi rata-rata 0,6703. Dengan demikian, model Piczak-CNN menunjukkan keunggulan akurasi relatif sebesar 30,6% dibandingkan PANNs CNN14.

Dari aspek efisiensi komputasional, model Piczak-CNN memiliki keuntungan signifikan. Model ini berukuran 205 MB dengan 26,3 juta parameter, sementara PANNs CNN14 berukuran 319 MB dengan 80,7 juta parameter. Berdasarkan perhitungan relatif, Piczak-CNN mengurangi ukuran model sebesar 35,7% dan jumlah parameter sebesar 67,4% dibandingkan PANNs CNN14. Temuan ini menunjukkan bahwa beban penyimpanan dan kompleksitas model Piczak-CNN lebih rendah, sehingga lebih sesuai untuk skenario perangkat dengan keterbatasan sumber daya.

Untuk memperkuat analisis, efisiensi juga dievaluasi menggunakan rasio kinerja terhadap kompleksitas model. Rasio akurasi per satu juta parameter untuk Piczak-CNN adalah 0,0318 (0,836/26,3), sedangkan PANNs CNN14 sebesar 0,0079 (0,64/80,7). Artinya, efisiensi kinerja per parameter Piczak-CNN sekitar 4,0 kali lebih baik. Rasio akurasi per MB model juga menunjukkan pola serupa, yaitu 0,0041 pada Piczak-CNN dan 0,0020 pada PANNs CNN14, sehingga efisiensi terhadap ukuran model meningkat sekitar 2,0 kali. Dengan demikian, hipotesis bahwa Piczak-CNN lebih efisien dibandingkan model pembanding didukung oleh indikator kuantitatif yang konsisten.

Meskipun demikian, penelitian ini belum melakukan pengukuran waktu inferensi end-to-end (dalam milidetik) pada perangkat target. Oleh karena itu, klaim efisiensi pada penelitian ini difokuskan pada efisiensi struktural model (ukuran berkas dan jumlah parameter) serta efisiensi kinerja relatif terhadap kompleksitas model.

Keunggulan tambahan model Piczak-CNN terletak pada spesialisasinya untuk keperluan domain tertentu. Model ini dirancang khusus untuk mengklasifikasikan lima kelas suara lingkungan yang relevan dengan penelitian ini, yakni klakson motor, klakson mobil, sirene, pecahan kaca, dan tangisan bayi. Spesialisasi ini memungkinkan model untuk membedakan dengan presisi tinggi antara suara-suara yang mirip namun memiliki karakteristik akustik yang berbeda. Sebaliknya, model PANNs CNN14 yang dilatih pada dataset AudioSet berukuran besar tidak memiliki fokus pada kelas-kelas spesifik ini, mengakibatkan performa yang lebih rendah pada tugas klasifikasi yang bersifat domain-spesifik.

Pengamatan empiris menunjukkan bahwa model Piczak-CNN mencapai konvergensi lebih cepat dibandingkan PANNs CNN14, terutama mengingat ukuran dataset yang terbatas (500 sampel total). Hasil evaluasi komparatif ini membuktikan bahwa model Piczak-CNN dengan konfigurasi *hyperparameter* optimal yang telah ditetapkan mengungguli model PANNs CNN14 dalam hal akurasi klasifikasi, efisiensi komputasional, dan kesesuaian terhadap karakteristik domain spesifik penelitian ini.

4.3 Pembahasan

Analisis hasil pengujian dari Tabel 4.2 dan Tabel 4.3 mengungkapkan bahwa laju pembelajaran (*learning rate*) memainkan peran krusial dalam meningkatkan performa model Piczak-CNN. Penurunan laju pembelajaran dari nilai awal yang lebih tinggi menghasilkan peningkatan akurasi yang signifikan, meskipun pengurangan yang berlebihan dapat menyebabkan konvergensi

lambat dan penurunan kinerja model.

Dalam konteks pemilihan algoritma optimisasi, SGD dengan momentum 0,9 dan laju pembelajaran 0,002 menunjukkan stabilitas dan akurasi yang jauh superior dibandingkan algoritma Adam, yang mengalami degradasi performa signifikan hingga mencapai akurasi 0,578 pada beberapa skenario pengujian. Hal tersebut mengindikasikan bahwa SGD lebih tepat untuk arsitektur Piczak-CNN dengan struktur sederhana, yaitu dua blok konvolusi dan pola ekstraksi fitur yang reguler.

Hasil eksperimen terhadap mekanisme *dropout* menunjukkan bahwa penurunan nilai *dropout* dari baseline 0,5 ke rentang 0,0–0,3 (dengan augmentasi tetap aktif) meningkatkan akurasi secara konsisten, dengan capaian terbaik 0,772 pada *dropout* 0,1 dibandingkan baseline 0,598. Temuan ini mengisyaratkan bahwa regularisasi *dropout* yang terlalu besar pada kondisi data penelitian dapat menghambat pembelajaran fitur yang relevan.

Sebaliknya, pengujian teknik augmentasi data menunjukkan bahwa penerapannya justru menurunkan performa model dibandingkan tanpa augmentasi. Akurasi dengan augmentasi berkisar 0,598–0,772, sementara tanpa augmentasi mencapai 0,812–0,822. Fenomena ini dapat dijelaskan oleh karakter dataset yang telah dikurasi secara konsisten dengan spesifikasi teknis yang seragam (laju sampel 44,1 kHz, format mono WAV, durasi standar), sehingga augmentasi justru menambahkan noise yang mengganggu pembelajaran representasi fitur audio asli. Sebagai tambahan, ukuran sampel yang relatif terbatas (500 total) membuat augmentasi agresif menghasilkan representasi yang divergen dari distribusi data asli.

Secara keseluruhan, konfigurasi optimal model Piczak-CNN ditetapkan sebagai berikut: *optimizer* SGD dengan laju pembelajaran 0,002 dan momentum 0,9, nilai *dropout rate* 0,0, serta tanpa penerapan augmentasi data. Model dengan konfigurasi tersebut mencapai akurasi rata-rata 0,823 dengan konsistensi tinggi pada ketiga percobaan verifikasi, membuktikan kapabilitas

generalisasi yang kuat dalam mengenali pola suara lingkungan kompleks mencakup variasi spektral dari klakson motor, klakson mobil, sirene, pecahan kaca, dan tangisan bayi. Keunggulan fundamental model ini terletak pada efisiensi arsitektur yang optimal namun tetap efektif dalam mengekstraksi fitur diskriminatif dari log-Mel spectrogram untuk tugas klasifikasi lima kelas target pada domain penelitian ini.

BAB V
KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan penelitian yang telah dilakukan, maka kesimpulan yang dapat diambil adalah sebagai berikut:

1. Optimalisasi arsitektur Piczak-CNN untuk klasifikasi suara lingkungan berorientasi keselamatan telah berhasil dicapai melalui *tuning hyperparameter* sistematis menggunakan skema *5-fold cross-validation* pada 16 kombinasi *hyperparameter*. Konfigurasi optimal menggunakan *optimizer* SGD (*learning rate* 0,002, momentum 0,9), *dropout rate* 0,0, dan tanpa augmentasi data menghasilkan akurasi rata-rata 83,6% dengan *F1-score* 0,81932 dan presisi 84,93%. Model ini terbukti efektif untuk mengklasifikasikan lima kelas suara peringatan (klakson mobil, klakson motor, sirene, pecahan kaca, dan tangisan bayi). Studi ablasi mengkonfirmasi bahwa penghilangan *dropout* dan augmentasi data dari konfigurasi *baseline* (akurasi 59,8%) meningkatkan performa secara substansial hingga 82,3%, sedangkan penggantian *optimizer* ke Adam menurunkan akurasi menjadi 72,2%.
2. Perbandingan performa model Piczak-CNN dengan model PANNs CNN14 yang di-*fine-tune* pada dataset yang sama menunjukkan keunggulan Piczak-CNN pada seluruh metrik evaluasi. Model Piczak-CNN mencapai akurasi 83,6%, *F1-score* 0,81932, dan presisi 84,93%, sedangkan PANNs CNN14 memperoleh akurasi 64%, *F1-score* 0,62322, dan presisi 67,03%. Dari sisi efisiensi komputasional, Piczak-CNN berukuran 205 MB dengan 26,3 juta parameter, lebih kecil dibandingkan PANNs CNN14 yang berukuran 319 MB dengan 80,7 juta parameter. Secara relatif, Piczak-CNN menurunkan ukuran model sebesar 35,7%

dan jumlah parameter sebesar 67,4%. Analisis rasio kinerja terhadap kompleksitas juga menunjukkan bahwa akurasi per satu juta parameter Piczak-CNN sekitar 4,0 kali lebih baik dibandingkan PANNs CNN14. Hasil ini mendukung hipotesis bahwa arsitektur yang lebih sederhana dan dioptimalkan secara spesifik untuk domain tertentu dapat mengungguli model *pre-trained* berskala besar pada klasifikasi suara lingkungan dengan dataset terbatas (500 sampel).

5.2 Saran

Adapun saran yang dapat diberikan untuk penelitian lebih lanjut berdasarkan penelitian ini adalah sebagai berikut:

1. Penelitian ini dapat diperluas dengan menggunakan dataset yang lebih besar dan lebih beragam untuk meningkatkan generalisasi model dalam klasifikasi suara lingkungan.
2. Penggunaan teknik augmentasi data yang lebih canggih dapat dipertimbangkan untuk meningkatkan performa model, terutama dalam menghadapi keterbatasan jumlah sampel dalam dataset.
3. Memperluas studi ablasi untuk mengevaluasi kontribusi masing-masing fitur dan lapisan dalam model dapat memberikan wawasan lebih dalam tentang bagaimana model memproses informasi suara lingkungan.
4. Model yang telah dioptimalkan dapat diimplementasikan pada perangkat *edge* untuk menguji performa *real-time* dalam situasi nyata, serta mengevaluasi dampaknya terhadap keselamatan bagi penyandang gangguan pendengaran.

DAFTAR PUSTAKA

[1] Fatih Demir, Daban Abdulsalam Abdullah, and Abdulkadir Sengur. "A New Deep CNN Model for Environmental Sound Classification". *IEEE Access* 8 (2020), pp. 66529–66537.

[2] Osman Salem, Ahmed Mehaoua, and Raouf Boutaba. "Vehicle Sound Recognition Assistance in IoT Systems for Hearing-Impaired Drivers". *IEEE Internet of Things Magazine* 8.4 (2025), pp. 14–21.

[3] C. Chen et al. "Audio-Based Early Warning System of Sound Events on the Road for Improving the Safety of Hearing-Impaired People". *2019 IEEE 8th Global Conference on Consumer Electronics (GCCE)*. 2019, pp. 933–936.

[4] T. Tasripan, H. Kusuma, and A. Huzaini. "Design and Implementation of Environmental Warning Systems for Drivers with Hearing Impairments". *European Journal of Electrical Engineering and Computer Science* 8.3 (2024).

[5] R. Kamelia and H. Kusuma. "Emergency Sound Classification and Visual Alert System for Enhanced Situational Awareness". *2024 International Conference on TVET Excellence & Development (ICTeD)*. 2024, pp. 213–218.

[6] T. Munasinghe and Y. Dulanjani. "AI Powered Context Aware Emergency Sound Recognition System for Enhancing Situational Awareness in Hearing Impaired Individuals". *2025 7th International Conference on Advancements in Computing (ICAC)*. 2025, pp. 1–6.

[7] S. Tan et al. "Emergency Sound Recognition and Direction Indication Using Machine Learning for Individuals with Hearing Loss". *Jurnal Kejuruteraan* (2025).

[8] Lujie Peng et al. “BSN-ESC: A Big–Small Network-Based Environmental Sound Classification Method for AIoT Applications”. *Sensors (Basel, Switzerland)* 23 (2023).

[9] Afsana Mou and M. Milanova. “Performance Analysis of Deep Learning Model-Compression Techniques for Audio Classification on Edge Devices”. *Sci* (2024).

[10] Karol J. Piczak. “Environmental sound classification with convolutional neural networks”. *2015 IEEE 25th International Workshop on Machine Learning for Signal Processing (MLSP)*. 2015, pp. 1–6.

[11] Zhichao Zhang et al. “Learning Attentive Representations for Environmental Sound Classification”. *IEEE Access* 7 (2019), pp. 130327–130339.

[12] Zhichao Zhang et al. “Deep Convolutional Neural Network with Mixup for Environmental Sound Classification” (2018), pp. 356–367.

[13] A. Bansal and N. K. Garg. “Environmental Sound Classification: A descriptive review of the literature”. *Elsevier B.V* (2022).

[14] S. Chandrakala and S. Jayalakshmi. “Environmental Audio Scene and Sound Event Recognition for Autonomous Surveillance”. *ACM Computing Surveys* 52.3 (2019), pp. 1–34.

[15] H. Arai et al. “A System that Warns of Dangerous Environmental Sounds for the Hearing Impaired”. *Proceedings of the Twelfth International Conference on Management Science and Engineering Management*. 2018, pp. 1043–1052.

[16] V. Shafiro et al. “Perception of Environmental Sounds by Experienced Cochlear Implant Patients”. *Ear and Hearing* 32.4 (2011), pp. 511–523.

[17] V.Wahyuningtyas. “Implementasi Ekstraksi Fitur untuk Klasifikasi Suara Urban menggunakan Deep Learning”. *Sains, Aplikasi, Komputasi dan Teknologi Informasi*. Vol. 3. 1. 2021, pp. 10–17.

[18] H Kusumah, N Nabbillah, and Audina.S. “Klasifikasi Kendaraan Berbasis Suara Di Lalu Lintas: Implementasi TinyML”. *Journal CERITA (Creative Education of Research in Information Technology and Artificial informatics)* 9.1 (2023).

[19] Bansal.A and Garg.N.K. “Environmental Sound Classification: A descriptive review of the literature”. *Intelligent Systems with Applications* 16 (2022), p. 200115. 2667-3053.

[20] Baljinder Kaur and Jaskirat Singh. “Audio Classification: Environmental sounds classification”. working paper or preprint. Dec. 2021.

[21] L. Wyse. *Audio Spectrogram Representations for Processing with Convolutional Neural Networks*. 2017. arXiv: 1706.09559 [cs.SD].

[22] Itsuki Toyoshima et al. “Multi-Input Speech Emotion Recognition Model Using Mel Spectrogram and GeMAPS”. *Sensors* 23.3 (2023). 1424-8220.

[23] I. N. Husna et al. “Rancang Bangun Sistem Deteksi Dan Perhitungan Jumlah Orang Menggunakan Metode Convolutional Neural Network (CNN)”. *Seminar Nasional Fortei Regional* 7 ().

[24] Go-Eun Woo et al. “Low-Complexity Hardware Architecture for Batch Normalization of CNN Training Accelerator”. *Computers, Materials & Continua* 84.2 (2025). 1 cites; Query date: 2025-12-04 01:09:23, pp. 3241–3257. 1546-2226.

[25] Qiuqiang Kong et al. “PANNs: Large-Scale Pretrained Audio Neural Networks for Audio Pattern Recognition”. *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 28 (2020), pp. 2880–2894.

[26] J. T. Townsend. “Theoretical analysis of an alphabetic confusion matrix”. *Perception & Psychophysics* 9.1 (1971), pp. 40–50. 1532-5962.

[27] Andi Lubis, Sevia Purnama, and Mas Afandi. “Sistem Pendeteksi Kantuk Berbasis Metode Haar Cascade Untuk Aplikasi Computer Vision”. *Techno.Com* 22 (Aug. 2023), pp. 589–598.

[28] S. Borstelmann and S. Jha. “Confusion in the Matrix: Going Beyond the Roc Curve”. *viXra* (2019).

[29] Kanae Takahashi et al. “Confidence interval for micro-averaged F1 and macro-averaged F1 scores”. *Applied Intelligence* 52.5 (2022), pp. 4961–4972. 1573-7497.

LAMPIRAN

A GitHub Proyek Tugas Akhir

GitHub proyek tugas akhir dapat diakses melalui tautan berikut :

https://github.com/mctosima/Jurnal-Klasifikasi-Suara-Lingkungan-Menggunakan-Piczak-